



POLITECNICO DI BARI

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
CORSO DI LAUREA IN INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

TESI DI LAUREA
IN
AUTOMAZIONE INDUSTRIALE

SISTEMI DI RETRIEVAL-AUGMENTED GENERATION:
CONFRONTO E INTEGRAZIONE DI TECNICHE DI
RICERCA VETTORIALE E LESSICALE PER LA
DIAGNOSTICA TECNICA

Relatore:
Prof. Ing. Agostino M. MANGINI

Correlatori:
Ing. Michele MARVULLI

Laureando: Daniele DI SPIRITO

Indice

Frontespizio	i
Liberatoria	iii
Liberatoria	iv
Dedica	vii
Ringraziamenti	ix
Indice	xiii
Indice delle figure	xv
Indice delle tabelle	xvii
Introduzione	xix
1 Stato dell'Arte e Riferimenti Teorici	1
1.1 Il contesto Industriale	1
1.2 Generative AI e Large Language Models (LLMs)	2
1.3 Limiti degli LLM	5
1.4 Il paradigma Retrieval-Augmented Generation (RAG)	6
1.4.1 Perché proprio RAG?	6
1.4.2 Pipeline di una RAG	7
1.4.2.1 Fase Off-line: Ingestion e Indicizzazione dei Dati	7
1.4.2.2 Fase On-line: La Pipeline RAG e l'Inferenza	7
1.5 Casi studio e progetti simili	8
2 Basi Teoriche e Concept Architetture	11
2.1 Teoria dell'Estrazione e Retrieval	11
2.1.1 Parsing e Data Cleaning	11
2.1.2 Chunking	12
2.1.3 Embedding	12
2.1.4 Vector Database e Geometria dello Spazio Semantico	13
2.2 Teoria della Generazione e Interazione	14
2.2.1 Meccanismi di Tokenizzazione e Algoritmi di Subword	14
2.2.2 Context Window e Iperparametri di Decodifica	15

2.2.3	Prompt Engineering, System Prompt e Agenti	16
2.3	Scelta dei Modelli e Deployment	16
2.3.1	Cloud vs On-Premise: il trade-off tra API e Privacy Aziendale	16
2.3.2	Selezione dei Modelli Linguistici di Riferimento	18
2.3.2.1	Evoluzioni dell'inferenza locale di frontiera — DwarfStar 4 (DS4)	18
2.3.3	Osservabilità e Tracciamento delle Richieste via LangSmith	19
2.4	Metriche di valutazione	19
2.4.1	Metriche di valutazione e Framework di Testing	19
2.4.1.1	Valutazione del Modulo di Retrieval	19
2.4.1.2	Valutazione della Generazione e Framework RAGAS	20
3	Sviluppo ed Implementazione	23
3.1	Pipeline di Ingestion e Indicizzazione	23
3.1.1	Document Parsing	23
3.1.2	Strategie di Chunking e Generazione degli Embedding	27
3.1.2.1	Analisi di Sensibilità al Chunk Size	28
3.1.3	Archiviazione e Struttura del Database Vettoriale	28
3.2	Strategie di Retrieval Avanzato	30
3.2.1	Retrieval Semantico Singolo (Dense Retrieval)	30
3.2.2	Retrieval Ibrido (Dense + Sparse/BM25)	30
3.2.2.1	La transizione teorica da TF-IDF a BM25	30
3.2.2.2	Il vincolo della lingua e il Translation Layer	32
3.2.2.3	Implementazione dell'Ensemble	32
3.2.3	Applicazione del Re-ranking	34
3.2.3.1	Il divario teorico tra Bi-Encoder e Cross-Encoder	34
3.2.3.2	Ingegnerizzazione del codice e gestione dei parametri k	36
3.2.4	Integrazione e Architettura GraphRAG	37
3.2.4.1	Topologia del Grafo Relazionale e Rilevamento delle Entità	39
3.2.4.2	Algoritmo di Espansione e Funzionamento del Codice	39
3.3	Prompt Engineering e Orchestrazione dei Modelli	43
3.3.1	Ingegnerizzazione dei Prompt e Regole di Grounding	43
3.3.1.1	Il Prompt di Classificazione degli Intenti (Query Routing)	43
3.3.1.2	Il System Prompt e le Regole di Ancoraggio Fattuale	45
3.3.2	Integrazione alle API Cloud (OpenRouter) e On-Premise (LM Studio)	46
3.3.3	Strumentazione di Telemetria e Monitoraggio con LangSmith	47
3.4	Interfaccia Utente e Gestione della Conversazione	48
3.4.1	Architettura e Presentazione dell'Applicazione Web	48
3.4.2	Gestione della Conversational History e Riconoscimento Vocale	49
3.4.2.1	Persistenza della Memoria e Caching delle Risorse	50
3.4.2.2	Integrazione del modulo Speech-to-Text	50
4	Sperimentazione, Valutazione e Analisi dei Risultati	53
4.1	Definizione e Validazione del Dataset di Test	53
4.1.1	Architettura della Pipeline di Generazione	53
4.1.2	Analisi Campionaria del Ground Truth Consolidato	54
4.2	Valutazione Quantitativa del Retrieval	55

4.3	Valutazione della Generazione e Metriche RAGAS	57
4.4	Studio Sperimentale sul Campo e Latenze Finali	63
4.4.1	Confronto con le Metodologie di Manutenzione Tradizionali	63
	Conclusioni	65
	Bibliografia	67

Indice delle figure

1.1	Fanuc R-30iB Mate (https://fanuc.com)	2
1.2	Tokenizer GPT-3 (https://platform.openai.com/tokenizer)	3
1.3	https://en.wikipedia.org/wiki/Word2vec	4
1.4	Pipeline di una RAG (off-line e on-line)	8
2.1	Schema di una Finestra di Contesto in un'architettura RAG	17
3.1	Istogramma Hit Rate ed MRR per metodo di parsing utilizzato	25
3.2	Analisi comparativa della latenza media di retrieval (espressa in secondi) per metodo di parsing utilizzato	26
3.3	Schema pipeline Naive RAG	31
3.4	Schema pipeline Hybrid RAG	35
3.5	Schema pipeline Rerank RAG	38
3.6	Mappa Relazionale globale del GraphRAG di ARIS. Si osservi la presenza di macro-cluster isolati che rispecchiano le singole sezioni della manualistica. . .	40
3.7	Dettaglio microscopico di un'espansione relazionale nel grafo per una sezione di connessioni (connections.pdf).	40
3.8	Schema pipeline Graph RAG	44
3.9	Interfaccia grafica dell'applicazione ARIS sviluppata in ambiente Streamlit. Sulla barra laterale sinistra sono visibili i moduli di configurazione iperparametrica, mentre a destra si sviluppa il pannello interattivo di messaggistica. .	49
4.1	Analisi comparativa dell'Hit Rate@3 e del Mean Reciprocal Rank (MRR) per le quattro strategie di retrieval al variare del parser e dell'ambiente (Query in lingua inglese).	56
4.2	Latenza computazionale (espressa in secondi) del solo processo di retrieval per le diverse configurazioni di database.	57
4.3	Analisi dell'Accuratezza di generazione (Hit Rate %) al variare dell'algoritmo RAG e dell'ambiente LLM, stratificata per lingua inglese (en) e italiana (it).	58
4.4	Istogrammi comparativi delle metriche RAGAS in ambiente Cloud per la lingua inglese e italiana.	60
4.5	Diagramma radar complessivo delle metriche di generazione per le quattro strategie RAG analizzate.	61
4.6	Scomposizione a griglia dei diagrammi radar per una valutazione isolata di ciascun algoritmo RAG.	62
4.7	Tempo di risposta totale percepito dall'utente (in secondi) sull'interfaccia Streamlit per le diverse strategie operative.	63

Indice delle tabelle

3.1	Analisi di sensibilità del Chunk Size applicata al Retrieval Vettoriale.	28
3.2	Analisi di sensibilità del Chunk Size applicata al Retrieval Ibrido.	28
4.1	Tassonomia e criteri di partizionamento iperparametrico del Dataset di Test.	54
4.2	Estratto campionario delle coppie di validazione componenti il Ground Truth finale del sistema ARIS.	55
4.3	Quadro comparativo sinottico tra la ricerca documentale classica e l'adozione della piattaforma ARIS.	64

Introduzione

Negli ultimi anni, l'avvento dei Large Language Model (LLM) ha cambiato radicalmente il panorama tecnologico globale. Tra le innovazioni più significative spicca l'integrazione di assistenti virtuali in grado di automatizzare compiti ripetitivi e a basso valore aggiunto.

Nell'ambito dell'Automazione Industriale, l'integrazione e la manutenzione di un braccio robotico richiedono uno studio approfondito e la memorizzazione di specifiche tecniche complesse, spesso frammentate in corposi manuali di riferimento. Nel contesto di questa tesi, tale problematica è stata affrontata prendendo come caso studio il controller industriale **Fanuc R-30iB Mate** (e la sua declinazione **Mate Plus**), una macchina caratterizzata da una vasta e dettagliata documentazione tecnica. Consultare manualmente questi testi risulta dispendioso in termini di tempo e incline all'errore umano, qualora si tenti di accelerare la ricerca delle informazioni. Lo scopo di questo lavoro consiste, pertanto, nella progettazione e nello sviluppo di un *chatbot* denominato ARIS¹, basato sull'implementazione di diverse architetture RAG² e specificamente istruito sulla manualistica del sistema Fanuc.

L'adozione di una *pipeline* RAG permette alle aziende di creare un ecosistema documentale robusto e resiliente rispetto alle innovazioni, sia in termini di risorse umane (agevolando l'ingresso di nuovi addetti), sia sul piano tecnologico (rendendo l'apprendimento dei nuovi macchinari graduale e sostenibile). L'introduzione di un simile assistente virtuale si rivela strategica anche nel contesto dell'attuale cambio generazionale nelle fabbriche, sollevando il personale esperto prossimo al pensionamento dall'onere di trasmettere manualmente il proprio sapere su ogni singolo impianto.

Questo lavoro di tesi non si limita alla realizzazione di un prototipo funzionante, ma si propone di confrontare diverse tecniche e combinazioni algoritmiche per individuare la configurazione capace di garantire la massima accuratezza. A tal fine, sono state implementate diverse metriche di valutazione quantitative (*score*) per ciascuno dei modelli generati. Tali metriche consentono un confronto oggettivo basato su parametri critici quali velocità di risposta, veridicità delle informazioni, accuratezza e costi computazionali, evidenziando i punti di forza che possono orientare la scelta dell'utente finale.

I test effettuati dimostrano che non esiste un modello universalmente superiore, poiché l'efficacia dipende strettamente dal contesto d'uso. Risulta fondamentale, infatti, selezionare l'architettura più idonea in base alla complessità dell'attività richiesta all'LLM. Per questa ragione, la soluzione proposta prevede un approccio flessibile che consente all'operatore di selezionare i modelli più performanti tra quelli testati, evitando lo spreco di risorse computazionali derivante dall'impiego di modelli sovradimensionati per compiti elementari.

L'obiettivo primario di questo elaborato consiste, in sintesi, nel *benchmarking* di differenti metodologie RAG (tra cui tradizionale, Ibrido, con Re-Ranking e GraphRAG), valutate sia

¹Assistente per Robot Industriale Smart

²Retrieval-Augmented Generation

in ambiente locale sia in *cloud*. L'analisi esplora l'integrazione di diversi LLM di base e la variazione dei principali iperparametri, al fine di individuare il *trade-off* ottimale tra qualità della generazione e costi di gestione.

Il presente elaborato è strutturato in quattro capitoli, organizzati come segue:

- Il **Capitolo 1 (Stato dell'Arte e Riferimenti Teorici)** inquadra il contesto industriale moderno e introduce i concetti fondamentali della Generative AI e dei Large Language Models, evidenziandone i limiti intrinseci e definendo il paradigma RAG come soluzione metodologica.
- Il **Capitolo 2 (Basi Teoriche e Concept Architetture)** approfondisce la teoria matematica e computazionale dietro i processi di estrazione (*retrieval*) e generazione, illustrando i criteri di scelta dei modelli, le strategie di *deployment* e le metriche usate per la valutazione del sistema.
- Il **Capitolo 3 (Sviluppo ed Implementazione)** descrive l'ingegnerizzazione vera e propria della *pipeline* di ARIS: dalla fase di *parsing* dei manuali del **Fanuc R-30iB Mate**, passando per il *chunking* semantico e la ricerca ibrida, fino al *prompt engineering* e alla creazione dell'interfaccia grafica in Streamlit ³.
- Il **Capitolo 4 (Sperimentazione, Valutazione e Analisi dei Risultati)** presenta la validazione empirica del sistema attraverso la definizione di un dataset di test dedicato. Vengono analizzati quantitativamente i risultati del *retrieval* e della generazione, concludendo con uno studio sperimentale sul campo per testare l'efficacia del chatbot in uno scenario industriale reale.

Infine, nelle **Conclusioni**, vengono riassunti i traguardi raggiunti, i limiti riscontrati e le possibili evoluzioni future del progetto.

³Framework open source di Python per la generazione semplice di siti web principalmente orientati alla *Data Science* (<https://streamlit.io>)

Capitolo 1

Stato dell'Arte e Riferimenti Teorici

1.1 Il contesto Industriale

Negli ultimi decenni, il settore manifatturiero è stato investito da un progresso tecnologico accelerato e costante, che ha ridefinito gli standard della produzione globale. Questa rapida evoluzione ha guidato la transizione verso sistemi di automazione industriale caratterizzati da una sofisticazione hardware e software senza precedenti. Se da un lato tale flessibilità incrementa l'efficienza dei processi, dall'altro aumenta esponenzialmente la complessità operativa, rendendo le attività di manutenzione, diagnostica e programmazione compiti estremamente ostici. Di conseguenza, il periodo di inserimento e formazione (*onboarding*) delle nuove risorse in azienda si è progressivamente dilatato, richiedendo lunghe fasi di studio dedicate alla consultazione di una manualistica tecnica che col tempo è divenuta imponente, stratificata e non sempre di immediata comprensione.

Le criticità di questo scenario si manifestano principalmente in due momenti chiave. In primo luogo, l'introduzione di un nuovo macchinario all'interno di una linea produttiva impone al personale di assimilare in tempi record procedure d'uso e di sicurezza radicalmente nuove, per non penalizzare la produttività dell'impianto. In secondo luogo, si riscontra la problematica cruciale del *troubleshooting*: l'insorgenza di un'anomalia richiede un intervento tempestivo per minimizzare i costi legati al fermo macchina (*downtime*). Tuttavia, la risoluzione dei guasti non è quasi mai immediata; spesso le informazioni necessarie sono sepolte all'interno di migliaia di pagine o richiedono una profonda conoscenza empirica del sistema, non direttamente accessibile tramite una consultazione sequenziale del testo.

A questo quadro già complesso si aggiunge una delicata sfida demografica nota in letteratura come **silver tsunami**, ovvero il progressivo e massiccio pensionamento della forza lavoro senior. Questo fenomeno espone le aziende al rischio concreto di perdere un patrimonio inestimabile di *know-how* pratico e competenze non codificate, prima che queste possano essere trasmesse alle nuove generazioni di operatori. Per arginare tali problematiche e governare la complessità, il paradigma dell'Industria 5.0 spinge verso l'adozione di tecnologie *human-centric*. In quest'ottica, diverse realtà industriali stanno integrando assistenti intelligenti basati sui recenti e straordinari progressi dei **Large Language Models** (LLM). Un simile strumento è in grado di simulare il comportamento di un tecnico esperto sempre disponibile sul campo, capace di dialogare in linguaggio naturale con l'operatore e di estrarre istantaneamente informazioni puntuali dal manuale digitale dell'impianto.

Come caso studio per la validazione di questo approccio, nel presente progetto di tesi è

stato preso in esame il controllore per bracci robotici **Fanuc R-30iB Mate** (nella sua declinazione **Mate Plus**). Questo sistema rappresenta uno standard industriale ampiamente diffuso nelle linee di montaggio, asservimento macchine e palettizzazione, grazie alle sue doti di compattezza, precisione e flessibilità architettuale. Il target è stato selezionato specificamente per l'elevata qualità e la completezza della sua documentazione ufficiale, la quale include una vasta e dettagliata classificazione dei codici di errore e delle relative procedure sequenziali (*step-by-step*) per la risoluzione dei guasti. Il **Fanuc R-30iB Mate** costituisce quindi il banco di prova ideale per testare la precisione di una *pipeline*¹ RAG, verificando se l'assistente virtuale sia in grado di mappare correttamente la sintomatologia dell'errore descritta dall'utente e di restituire la specifica procedura di ripristino prevista dal costruttore.



Figura 1.1: *Fanuc R-30iB Mate* (<https://fanuc.com>)

1.2 Generative AI e Large Language Models (LLMs)

La progettazione di un assistente virtuale intelligente si colloca all'interno del vasto campo dell'Intelligenza Artificiale (AI). L'AI rappresenta la branca della *Computer Science* che si occupa dello sviluppo di sistemi e algoritmi capaci di simulare funzioni cognitive umane, risolvendo compiti complessi attraverso l'apprendimento e il ragionamento. Sebbene le radici teoriche dell'AI risalgano agli anni '40 e '50 del Novecento - grazie alle intuizioni pionieristiche di Alan Turing - la sua integrazione pervasiva nel tessuto tecnologico e industriale quotidiano è avvenuta nell'ultimo decennio, trainata dall'incremento della potenza computazionale e dalla disponibilità massiva di dati (*Big Data*).

L'universo dell'Intelligenza Artificiale racchiude in sé molteplici sotto-paradigmi. Il nucleo tecnologico su cui si fonda questo lavoro di tesi è la sotto-categoria della **Generative AI** (AI Generativa). A differenza dei modelli predittivi o di classificazione tradizionali, i sistemi generativi sfruttano reti neurali profonde e con miliardi di parametri per sintetizzare nuovi contenuti multimediali, quali immagini, audio, codice sorgente e testo, partendo da un input in linguaggio naturale (*prompt*).

¹sequenza di elementi/azioni da svolgere in serie

Quando l'AI generativa è specializzata nell'elaborazione e nella produzione di contenuti testuali, si entra nel dominio dei *Language Models* (LM). Qualora la dimensione di tali modelli, espressa in termini di parametri architetturali e volume del dataset di addestramento, superi determinate scale critiche, si parla propriamente di **Large Language Models** (LLMs). Un LLM viene addestrato tramite computazione auto-supervisionata su un immenso corpus di documenti digitali estratti dal web e da librerie specializzate. Grazie a questa mole di informazioni, il processo di generazione testuale non si configura come una mera autocompilazione statistica o una predizione lineare della parola successiva; al contrario, il modello sviluppa una rappresentazione concettuale latente delle relazioni linguistiche, risultando capace di generare risposte coerenti, argomentate e dotate di una precisa struttura logico-semantica.

La capacità di un Large Language Model di comprendere la logica intercorrente tra le parole e di mappare il significato semantico di una frase non deriva da una comprensione biologica o concettuale, bensì dall'applicazione sinergica di raffinate tecniche matematiche e geometriche. Il flusso di elaborazione dell'informazione, dall'input testuale dell'utente alla generazione della risposta, è governato da una serie di pilastri architetturali e parametri computazionali descritti di seguito:

- **Tokenizzazione:** Rappresenta la fase di scomposizione preliminare del testo stringa in unità atomiche di calcolo, denominate *token*. Gli LLM non elaborano intere parole o singoli caratteri, ma sottostringhe frequenti (tramite algoritmi come il *Byte-Pair Encoding* [1]). Questo approccio permette al modello di gestire in modo efficiente neologismi, errori ortografici e termini tecnici tipici del lessico industriale, riducendo drasticamente la dimensione del vocabolario complessivo.

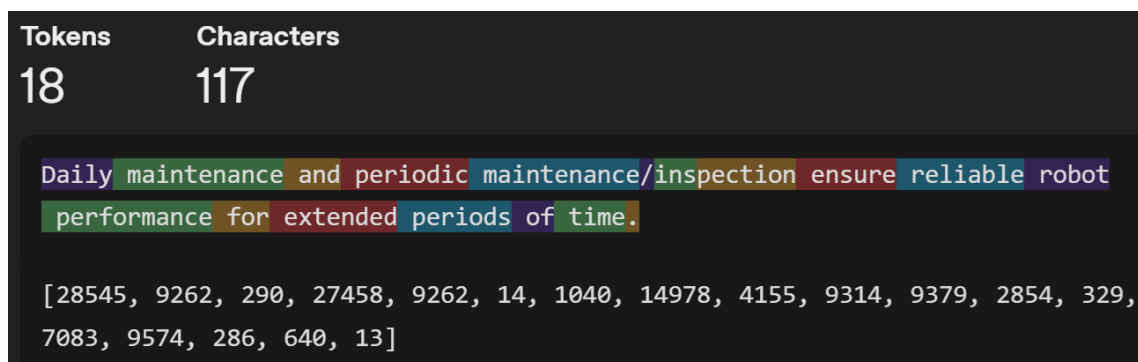


Figura 1.2: Tokenizer GPT-3 (<https://platform.openai.com/tokenizer>)

- **Embeddings:** Una volta convertito il testo in token, ciascuno di essi viene mappato in un vettore numerico ad alta dimensionalità all'interno di uno spazio vettoriale continuo. Questi vettori, chiamati *embeddings*, catturano il significato semantico e le relazioni sintattiche tra i termini. In questo spazio, concetti matematicamente vicini corrispondono a parole semanticamente correlate (ad esempio, i vettori di "robot" e "automa" registreranno una distanza euclidea o una similitudine cosinusoidale molto ridotta).

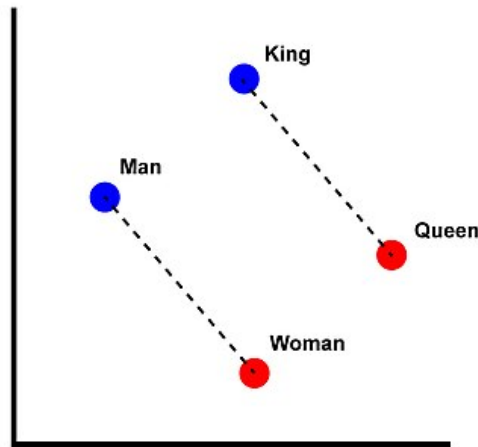


Figura 1.3: <https://en.wikipedia.org/wiki/Word2vec>

- **Self-Attention [2] (Meccanismo di Auto-Attenzione):** Rappresenta il vero motore concettuale del Transformer. Questo meccanismo consente al modello, durante l'elaborazione di uno specifico token, di calcolare dinamicamente un punteggio di rilevanza (*attention weight*) rispetto a tutti gli altri token presenti nel testo. Ciò permette di contestualizzare il significato di ogni parola in base all'intero periodo; ad esempio, consente all'algoritmo di discernere istantaneamente il significato della parola "asse" in un contesto geometrico rispetto a uno meccanico.
- **Positional Encoding (Codifica Posizionale):** Poiché l'architettura Transformer elabora tutti i token di una sequenza simultaneamente in parallelo (a differenza delle architetture ricorrenti sequenziali), essa non possiede una nozione intrinseca dell'ordine delle parole. Il *Positional Encoding* [2] ovvia a questo limite iniettando in ciascun vettore di embedding un'informazione matematica aggiuntiva legata alla posizione assoluta e relativa del token all'interno della frase, preservando l'ordine sintattico fondamentale per il senso del discorso.
- **Finestra di Contesto (*Context Window*):** Costituisce la quantità massima di dati (espressa in numero di token) che il modello è in grado di elaborare ed allocare in memoria in una singola iterazione, comprensiva sia del prompt di input che della risposta generata. Questo limite fisico rappresenta una barriera critica per l'elaborazione di manuali industriali di grandi dimensioni e costituisce il fulcro motivazionale per l'adozione di architetture di recupero esterne.
- **Temperatura:** È un iperparametro di configurazione che agisce in fase di inferenza sulla distribuzione di probabilità dei token in uscita. Un valore di temperatura tendente

a zero rende il modello deterministico e focalizzato sulle parole statisticamente più probabili, riducendo la variabilità e la creatività; al contrario, valori prossimi a uno aumentano la casualità della generazione. Nell'ambito del troubleshooting industriale, tale parametro viene mantenuto estremamente basso per garantire risposte rigorose e fedeli alla documentazione ufficiale.

- **Nucleus Sampling (Top-P):** È una tecnica di campionamento probabilistico utilizzata per controllare il processo di generazione testuale. Invece di considerare l'intero vocabolario, il modello ordina le potenziali parole successive in base alla loro probabilità e seleziona solo il sottoinsieme più ristretto (*nucleus*) la cui somma di probabilità cumulative soddisfa la soglia definita dal parametro p (ad esempio, il top 90%). Questo meccanismo lavora in sinergia con la temperatura per incrementare la fluidità e la coerenza del testo generato, scartando a priori risposte semanticamente sconnesse.

1.3 Limiti degli LLM

I Large Language Models rappresentano uno strumento straordinariamente potente, la cui evoluzione è stata guidata dall'aumento esponenziale della mole di dati e dei parametri di rete utilizzati durante la fase di addestramento (*training*). Tuttavia, tali modelli soffrono di limitazioni tecniche intrinseche che devono essere rigorosamente valutate prima di una loro integrazione in un contesto industriale sotto forma di assistente virtuale.

Queste criticità possono essere riassunte nei seguenti punti:

- **Allucinazioni [3]:** A causa della loro natura probabilistica e generativa, gli LLM presentano una tendenza non trascurabile a generare informazioni fattualmente errate, pur se sintatticamente impeccabili e apparentemente convincenti. Nell'ambito dell'automazione industriale, tale comportamento costituisce un rischio critico per la sicurezza. Questo problema è mitigabile impostando il parametro della temperatura a zero (0.0) — per rendere la generazione deterministica — e strutturando un *system prompt* rigido, che vincoli il modello a dichiarare la propria ignoranza qualora non disponga di dati certi.
- **Mancanza di conoscenza di dominio specifico:** Gli LLM commerciali vengono addestrati su *corpus* testuali generalisti estratti dal web. La conoscenza risultante è pertanto di tipo prevalentemente orizzontale, concentrata su tematiche globali e priva di verticalizzazioni specialistiche. Di conseguenza, un modello *stand-alone*² non è intrinsecamente in grado di conoscere le procedure di manutenzione specifiche o i codici di errore di una determinata macchina industriale. Per superare tale limite, si rende necessario l'accoppiamento con l'architettura RAG, descritta approfonditamente nei paragrafi successivi.
- **Mancanza di aggiornamento temporale (*Knowledge Cutoff*):** La conoscenza parametrica di un LLM rimane cristallizzata al momento in cui si conclude il processo di addestramento primario. Durante la fase di inferenza — ovvero il processo computazionale in cui il modello genera la risposta token per token — la rete neurale non

²utilizzato senza tecnologie di supporto

apprende nuove informazioni, ma si limita a elaborare l'input basandosi sui pesi sinaptici già consolidati. Un modello, pertanto, ignorerà qualsiasi evoluzione tecnologica, aggiornamento normativo o rilascio di nuovi firmware successivi alla propria data di *cutoff*.

- **Finestra di contesto limitata:** La finestra di contesto, che delimita il numero massimo di token allocabili simultaneamente tra input e output, rappresenta un vincolo fisico invalicabile. Questa limitazione impedisce l'inserimento diretto di un intero manuale tecnico all'interno del *prompt* per fornire al modello una conoscenza temporanea. Il tentativo di caricare una documentazione imponente provocherebbe il superamento immediato del limite di token consentito o, nei modelli a contesto esteso, una marcata degradazione delle performance di recupero informativo.

1.4 Il paradigma Retrieval-Augmented Generation (RAG)

1.4.1 Perché proprio RAG?

Il paradigma *Retrieval-Augmented Generation* (RAG) rappresenta un'architettura ibrida che estende le capacità native di un Large Language Model consentendogli di consultare fonti informative esterne e dinamiche prima di avviare il processo di generazione testuale. In ambito industriale, questa tecnica si rivela cruciale: permette infatti di interfacciare il modello con la documentazione tecnica proprietaria, manuali d'uso o database aziendali contenenti dati privati e sensibili. Un vantaggio strategico fondamentale risiede nella sicurezza dei dati: l'architettura RAG evita l'esposizione diretta di interi patrimoni documentali verso l'esterno, mitigando i rischi di violazione della privacy legati all'uso di modelli commerciali in *cloud* attraverso l'iniezione dinamica e mirata dei soli frammenti di testo necessari.

L'obiettivo primario della RAG è l'ancoraggio (*grounding*) della risposta a fatti verificabili. Attraverso la fase di *Retrieval* (recupero), il sistema seleziona esclusivamente le fonti più pertinenti alla richiesta dell'utente e le inserisce all'interno della *context window* dell'LLM. Sulla base di questo contesto informativo circoscritto, il modello opera non più basandosi sulla propria memoria probabilistica, ma eseguendo un compito di sintesi e rielaborazione guidata, azzerando quasi completamente il rischio di allucinazioni fattuali.

Per espandere la conoscenza di un LLM, la RAG non costituisce l'unica opzione metodologica; l'alternativa classica è rappresentata dal *Fine-Tuning*, ovvero il riaddestramento parziale dei pesi sinaptici della rete neurale su un dataset verticale specifico. Tuttavia, per compiti di consultazione documentale, il *Fine-Tuning* è stato progressivamente affiancato o sostituito dai sistemi RAG a causa di precise limitazioni: richiede infrastrutture computazionali estremamente onerose, non garantisce l'eliminazione delle allucinazioni e non permette un aggiornamento in tempo reale dei dati (richiedendo un nuovo ciclo di addestramento per ogni modifica documentale). In termini teorici, il *Fine-Tuning* modifica la **conoscenza parametrica** del modello, intervenendo sul suo "comportamento", sul tono o sullo stile espositivo. La RAG, al contrario, agisce sulla **conoscenza non parametrica**, configurandosi come la soluzione ideale qualora si disponga di una base di conoscenza strutturata e soggetta a frequenti aggiornamenti, come la manualistica del controller **Fanuc R-30iB Mate**.

1.4.2 Pipeline di una RAG

L'architettura complessiva di un sistema RAG non deve essere considerata come un unico processo lineare, bensì come l'unione di due macro-pipeline concettualmente e temporalmente distinte: una fase di **preparazione e indicizzazione dei dati (fase off-line)** e una fase di **esecuzione e inferenza (fase on-line)**.

Di seguito viene descritta la scomposizione logica di queste due componenti fondamentali:

1.4.2.1 Fase Off-line: Ingestion e Indicizzazione dei Dati

Questa prima macro-fase viene eseguita in modalità *one-time* (una tantum) o periodicamente per popolare la base di conoscenza del sistema, convertendo la manualistica tecnica in un formato digeribile dall'algoritmo:

1. **Estrazione (*Ingestion*):** Consiste nel recupero dei documenti dai formati sorgente (quali PDF tecnici, fogli di calcolo o database). Questa fase può essere bypassata o semplificata qualora il materiale di partenza sia già archiviato sotto forma di file di testo strutturato (come Markdown `.md` o file `.txt`).
2. **Preprocessing e Data Cleaning:** È la fase dedicata alla normalizzazione del testo estratto. In questo step si procede alla rimozione di rumore algoritmico, caratteri speciali residui del *parsing*, formattazioni errate o elementi grafici non testuali, standardizzando il testo per le successive computazioni.
3. **Chunking (Segmentazione):** Poiché i manuali industriali presentano dimensioni imponenti che eccedono i limiti fisici ed economici della *context window*, il testo pulito viene frammentato in porzioni atomiche chiamate *chunk*. La segmentazione può seguire logiche fisse (basate sul numero di token) o logiche avanzate e semantiche, preservando l'integrità logica di capitoli, paragrafi o tabelle.
4. **Embedding:** Ciascun *chunk* testuale, privo di valore matematico intrinseco, viene convertito in un vettore numerico denso ad alta dimensionalità attraverso un modello di *Text Embedding*. Questo passaggio proietta il testo all'interno di uno spazio geometrico capace di codificarne rigidamente il significato semantico.

1.4.2.2 Fase On-line: La Pipeline RAG e l'Inferenza

Questa seconda macro-fase rappresenta il cuore operativo del sistema in tempo reale. Si attiva ogni qualvolta un operatore sul campo formula una richiesta o descrive un'anomalia del robot:

- **Retrieval (Recupero Semantico):** La query dell'utente viene istantaneamente convertita in embedding. Il sistema interroga il *Vector Database* calcolando la similitudine geometrica (es. la *Cosine Similarity*) tra il vettore della query e quelli dei chunk memorizzati, estraendo solo i frammenti di testo del manuale Fanuc statisticamente più pertinenti.
- **Augmentation e Generazione:** I chunk recuperati vengono iniettati dinamicamente all'interno del prompt d'istruzione (*system prompt*). L'LLM riceve così questa finestra di contesto arricchita e, sfruttando le sue capacità di *In-Context Learning*³, sintetizza

³L'abilità di un LLM di comprendere e apprendere il contenuto della query

una risposta accurata, verificabile e rigidamente ancorata alla documentazione ufficiale, minimizzando il rischio di allucinazioni.

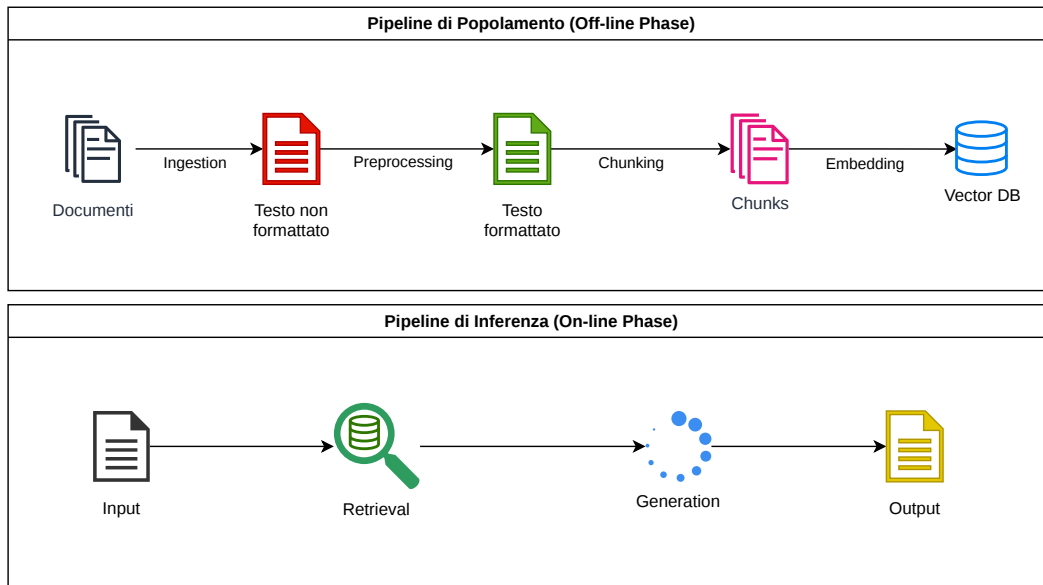


Figura 1.4: Pipeline di una RAG (off-line e on-line)

1.5 Casi studio e progetti simili

Per contestualizzare l'originalità e la solidità metodologica del sistema ARIS, è necessario analizzare lo stato dell'arte relativo all'applicazione dei sistemi RAG in contesti industriali e di manutenzione critica. La letteratura scientifica recente offre diversi framework che affrontano le medesime sfide core di questo elaborato — quali l'adattamento al dominio, la sicurezza della generazione e la gestione di documentazione complessa — fornendo importanti punti di riferimento architetturali.

- LLM-R: A Framework for Domain-Adaptive Maintenance Scheme Generation [4]** Il framework *LLM-R* si concentra specificamente sull'ottimizzazione della generazione di schemi di manutenzione industriale attraverso l'uso di LLM adattati al dominio d'applicazione. Gli autori evidenziano come i modelli commerciali standard falliscano nell'identificare le procedure corrette a causa della specificità del lessico ingegneristico. L'aspetto più rilevante di *LLM-R*, che ha ispirato la progettazione di ARIS, è la dimostrazione di come una pipeline di recupero mirata superi in accuratezza ed efficienza economica un approccio basato sul fine-tuning completo del modello. Questo caso studio valida la scelta di utilizzare la conoscenza non parametrica per mappare la sintomatologia dei guasti, fornendo la base teorica per l'applicazione degli LLM alla manutenzione dei bracci robotici.
- RAGuard: A Novel Approach for In-Context Safe Retrieval Augmented Generation [5]** Nel contesto dell'automazione industriale, l'affidabilità dell'output è un vincolo mandatorio: una risposta errata o un'allucinazione sul significato di un codice

d'errore può compromettere l'integrità fisica del macchinario o la sicurezza dell'operatore. Il progetto *RAGuard* affronta direttamente questa problematica, introducendo un framework per il controllo di sicurezza *in-context* all'interno delle pipeline RAG. RAGuard implementa uno strato di protezione che valuta la coerenza tra i chunk recuperati dal database e la risposta finale dell'LLM, bloccando o correggendo l'output qualora vengano rilevate derive informative o risposte non strettamente ancorate alla fonte. Questo approccio ha guidato lo sviluppo di ARIS nella definizione di *system prompt* restrittivi e nell'ottimizzazione dei parametri di temperatura, enfatizzando la necessità di un assistente virtuale "sicuro per il posizionamento in fabbrica".

- **LARAG: Link-Aware Retrieval Strategy for RAG Systems in Hyperlinked Technical Documentation [6]** La manualistica dei moderni controller industriali, come quella del **Fanuc R-30iB Mate**, non è un semplice testo lineare, ma una rete complessa di informazioni ricca di tabelle di errore, rimandi incrociati ad appendici e collegamenti ipertestuali. I sistemi RAG tradizionali spesso falliscono in questo scenario poiché il *chunking* lineare spezza i legami logici tra le pagine. Il framework *LARAG* risolve questa criticità introducendo una strategia di recupero "consapevole dei collegamenti" (*Link-Aware*). LARAG non analizza i chunk in modo isolato, ma mappa la struttura ipertestuale della documentazione tecnica, garantendo che se viene recuperato un determinato paragrafo, il sistema includa nel contesto anche le tabelle o le appendici a esso collegate. Questo studio rappresenta il punto di giunzione ideale con le tecniche avanzate analizzate in questa tesi, offrendo solide motivazioni teoriche per l'introduzione di architetture di tipo **GraphRAG** e di **Re-Ranking** per la gestione dei manuali Fanuc.

Capitolo 2

Basi Teoriche e Concept Architetture

2.1 Teoria dell'Estrazione e Retrieval

2.1.1 Parsing e Data Cleaning

Nel contesto di una *pipeline* di popolamento documentale per sistemi RAG, la fase di estrazione costituisce il primo e più critico blocco operativo. La stragrande maggioranza della manualistica tecnica industriale è archiviata in formato **PDF** (Portable Document Format); tuttavia, l'apparente uniformità di questa estensione cela differenze strutturali profonde che impongono logiche di *parsing* (estrazione e analisi sintattica) radicalmente diverse:

1. **PDF Nativi (Digitali)**: Sono documenti generati direttamente da software di videoscrittura o CAD editi dai costruttori. In questo scenario, i caratteri e la formattazione sono mappati internamente come metadati testuali espliciti. Il *parser* può intercettare programmaticamente i vettori di testo e preservarne l'ordine logico di lettura e la gerarchia dei capitoli senza alterazioni geometriche.
2. **PDF Scansionati (Raster)**: Sono il risultato della digitalizzazione ottica di documenti cartacei e si configurano, a livello logico, come pure matrici di pixel (immagini). In questo caso, l'estrazione richiede l'impiego di motori **OCR** (*Optical Character Recognition*), algoritmi di visione computazionale atti a riconoscere i glifi all'interno dei layer grafici e a ricostruirne i caratteri testuali equivalenti, con un costo computazionale superiore e un margine di errore legato alla qualità della scansione.

La documentazione ufficiale del controller **Fanuc R-30iB Mate** afferisce alla prima categoria, presentandosi come un ecosistema di PDF nativi ad alta definizione. Nonostante l'assenza di barriere legate all'OCR¹, il *parsing* solleva complesse sfide ingegneristiche, prime fra tutte la **gestione delle strutture tabellari**. I manuali del sistema Fanuc contengono ampie tabelle dedicate alla diagnostica e alla mappatura sequenziale dei codici di errore; estrarre questi blocchi in modo lineare distruggerebbe la correlazione bidimensionale tra righe e colonne, invalidando la semantica del dato. Diventa quindi obbligatorio l'utilizzo di algoritmi di *Layout Analysis* capaci di preservare la struttura tabellare (ad esempio convertendola in formati semantici come il *Markdown*² o *JSON*³).

¹Optical Character Recognition

²linguaggio markup per la formattazione di testi

³JavaScript Object Notation

Una volta isolati i flussi testuali grezzi, si accede alla sotto-fase di **Data Cleaning** (normalizzazione e pulizia del dato). Il testo estratto è intrinsecamente affetto da rumore algoritmico: stringhe isolate dovute a numeri di pagina, intestazioni ripetitive di capitolo (*headers* e *footers*) ed elementi testuali orfani estratti dall'interno di schemi elettrici o figure geometriche. Questo rumore viene rimosso mediante l'applicazione di filtri e costrutti a espressioni regolari (*Regex*). Al contrario, le didascalie dei grafici e dei diagrammi tecnici vengono accuratamente preservate e correlate al contesto, in quanto vettori fondamentali di informazioni esplicative sulle logiche di funzionamento della macchina.

2.1.2 Chunking

Disponendo di un flusso testuale normalizzato e strutturato per ciascun documento, l'approccio più immediato consisterebbe nell'iniettare l'intero contenuto direttamente all'interno del prompt d'inferenza. Tuttavia, come analizzato nei paragrafi precedenti, questa strategia si scontra con i limiti fisici della *context window* e con le degradazioni prestazionali derivanti dal fenomeno del *Lost-in-the-Middle*, che porta il modello a ignorare o a pesare in misura minore le informazioni collocate nella parte centrale di contesti eccessivamente estesi.

Si rende pertanto necessario implementare una strategia di **Chunking** (segmentazione del testo), finalizzata a suddividere l'opera documentale in frammenti atomici dotati di coerenza logica e semantica. All'interno dello stato dell'arte, questa operazione di partizionamento viene eseguita principalmente secondo due metodologie:

- **Chunking a dimensione fissa (basato sul conteggio dei Token):** Questa tecnica prevede la scomposizione del testo in blocchi di lunghezza predefinita espressa in numero di token (ad esempio, chunk da 512 o 1024 token). Per evitare che concetti chiave o frasi complesse vengano troncati a metà inficiando la comprensione dell'LLM, a questa metodologia si associa quasi sempre un parametro di *Overlap* [7] (sovrapposizione). L'overlap consiste nel replicare una percentuale di token del chunk precedente all'inizio del successivo (tipicamente il 10-20%), garantendo la continuità del contesto semantico tra i blocchi adiacenti.
- **Chunking strutturale o ricorsivo (basato sulla sintassi del documento):** Questo approccio, più raffinato ed efficace per la manualistica tecnica, sfrutta la formattazione nativa del testo (come i tag Markdown o i caratteri di interruzione di riga) per segmentare il documento in corrispondenza dei confini logici naturali, quali capitoli, paragrafi, sezioni o singole voci di una tabella di errore. In questo modo si preserva nativamente l'integrità concettuale delle istruzioni del costruttore.

Una volta ottenuti i singoli chunk, l'architettura RAG necessita di un meccanismo per confrontare matematicamente la richiesta dell'operatore con i frammenti di testo isolati. Questo passaggio introduce la necessità di proiettare il testo all'interno di uno spazio geometrico condiviso attraverso la teoria degli **Embedding Vettoriali**.

2.1.3 Embedding

Il processo di *Embedding* costituisce il ponte algoritmico tra il linguaggio naturale non strutturato e la computazione matematica ed è la componente che permette di quantificare la

semantica di un *chunk* di testo. Storicamente, l'evoluzione di questa tecnologia ha segnato il passaggio da rappresentazioni puramente lessicali a sofisticati spazi semantici densi.

Le prime tecniche di elaborazione del linguaggio naturale si basavano su approcci a codifica sparsa (*One-Hot Encoding*) o puramente statistici (come la matrice TF-IDF⁴), in cui ogni parola era mappata come un vettore ortogonale indipendente. Questo paradigma presentava due limiti fondamentali: l'assenza totale di relazioni semantiche tra i vettori (ad esempio, le parole "robot" e "automa" risultavano geometricamente distanti tanto quanto "robot" e "mela") e una dimensionalità insostenibile al crescere del vocabolario.

Il primo vero cambio di paradigma è avvenuto nel 2013 con l'introduzione di **Word2Vec** [8], un'architettura di rete neurale leggera capace di mappare le parole in vettori densi a bassa dimensionalità basandosi sull'ipotesi distribuzionale, secondo cui parole che compaiono in contesti simili tendono ad avere significati simili. Nonostante il successo, Word2Vec e i successivi modelli come GloVe soffrivano di un limite strutturale: erano modelli statici. Una parola riceveva un unico vettore di embedding a prescindere dal contesto in cui si trovava, fallendo sistematicamente nella gestione dei termini polisemici.

I modelli di embedding densi attuali, basati sull'architettura dei *Transformer* [2] (come i modelli della famiglia BERT⁵ o gli embedding di OpenAI e Cohere), hanno superato questa barriera introducendo gli **embedding contestualizzati**. In queste architetture, il vettore associato a un singolo token non è statico, ma viene calcolato dinamicamente integrando le informazioni fornite da tutti gli altri token presenti nella sequenza. Di conseguenza, il medesimo termine tecnico presente nei manuali Fanuc assumerà coordinate geometriche differenti nello spazio vettoriale a seconda che sia inserito in una procedura di diagnostica software o in una specifica di manutenzione meccanica, garantendo una precisione semantica senza precedenti nella pipeline di recupero.

Dal punto di vista computazionale, mentre l'embedding di un singolo token corrisponde all'estrazione di un vettore riga da una matrice di look-up, la generazione dell'embedding di un'intera frase (o *chunk*) richiede un'operazione di aggregazione nota come **Pooling**.

Nelle moderne architetture basate su Transformer (come i modelli *Sentence-Transformers*), la sequenza di input viene elaborata in parallelo attraverso i layer di *self-attention*, producendo in output una matrice di vettori in cui ciascun token ha incorporato il contesto circostante. Per collassare questa matrice in un unico vettore denso rappresentativo dell'intero documento, si applica prevalentemente il *Mean Pooling*, ovvero la media aritmetica elemento per elemento dei vettori di tutti i token. In alternativa, in alcuni modelli orientati alla classificazione, viene estratto esclusivamente il vettore associato al token speciale [CLS], posizionato all'inizio della sequenza e deputato ad accumulare l'informazione semantica globale durante la fase di *forward pass*.

2.1.4 Vector Database e Geometria dello Spazio Semantico

Una volta convertiti i *chunk* documentali in vettori densi ad alta dimensionalità, si rende necessaria un'infrastruttura software ottimizzata per la loro memorizzazione, indicizzazione e ricerca rapida: il **Vector Database** (Vector DB).

A differenza dei database relazionali tradizionali, progettati per interrogazioni puntuali basate su stringhe o chiavi esplicite, un database vettoriale opera all'interno di uno **spazio**

⁴Term Frequency - Inverse Document Frequency

⁵Bidirectional Encoder Representations from Transformers

semantico. Questo spazio può essere immaginato come un sistema di coordinate geometriche multi-dimensionale (spesso compreso tra le 768 e le 1536 dimensioni, a seconda del modello di embedding utilizzato). All'interno di questa topologia, la vicinanza spaziale tra due punti esprime direttamente il grado di affinità concettuale dei testi che essi rappresentano.

Il compito principale del Vector DB all'interno di una pipeline RAG consiste nel ricevere la query inserita dall'operatore, convertirla in un vettore di ricerca mediante lo stesso modello di embedding e identificare i vettori d'archivio geometricamente più prossimi. Per determinare matematicamente tale vicinanza senza risentire della differente lunghezza dei testi, la metrica standard più robusta e ampiamente utilizzata è la **Similitudine Coseno** (*Cosine Similarity*).

La Cosine Similarity non misura la distanza geometrica lineare (euclidea) tra due punti, bensì l'angolo compreso tra i due vettori direzionali nello spazio multi-dimensionale. Dati due vettori A e B , la formula matematica per il calcolo della similitudine è definita come il rapporto tra il loro prodotto scalare e il prodotto delle loro norme euclidee:

$$\text{CosineSimilarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.1)$$

Il risultato di questa funzione è un valore scalare strettamente confinato nell'intervallo $[-1, 1]$:

- Un punteggio pari a 1 indica che i due vettori sono perfettamente sovrapposti (angolo $\theta = 0^\circ$), denotando la massima identità semantica.
- Un punteggio pari a 0 indica ortogonalità tra i vettori (angolo $\theta = 90^\circ$), che si traduce in una totale indipendenza concettuale tra la query e il testo esaminato.
- Un punteggio pari a -1 indica vettori diametralmente opposti (angolo $\theta = 180^\circ$).

Durante la fase di *Retrieval*, il Vector DB esegue questo calcolo su larga scala, estraendo i primi k chunk (tecnica nota come *Top-k Retrieval*) che registrano il valore di Cosine Similarity più elevato rispetto alla query dell'utente, pronti per essere veicolati nel prompt finale dell'LLM.

2.2 Teoria della Generazione e Interazione

2.2.1 Meccanismi di Tokenizzazione e Algoritmi di Subword

La fase di generazione e comprensione del testo da parte di un Large Language Model richiede la conversione preventiva dei caratteri alfabetici in indici numerici discreti fruibili dalle reti neurali. Questo processo prende il nome di **Tokenizzazione**. Storicamente, i primi sistemi di NLP adottavano una tokenizzazione a livello di parola (*Word-based*), esponendosi al problema dei termini fuori vocabolario (*Out-Of-Vocabulary*, OOV) davanti a neologismi o errori ortografici, o a livello di singolo carattere (*Character-based*), la quale frammentava eccessivamente l'informazione perdendo le relazioni semantiche a lungo raggio.

Per superare queste barriere, gli LLM moderni adottano algoritmi di scomposizione in sotto-parole (**Subword Tokenization**). Tra i più diffusi nello stato dell'arte spicca il **Byte-Pair Encoding (BPE)** [1] (utilizzato da modelli come la famiglia GPT e Llama). Il funzionamento del BPE si articola in una fase di addestramento statistico e una di inferenza:

1. Si inizializza il vocabolario inserendo tutti i singoli caratteri e simboli base presenti nel dataset di addestramento.
2. Il testo viene esaminato iterativamente alla ricerca della coppia di byte (o sotto-stringhe) consecutiva che registra la frequenza di co-occorrenza più elevata.
3. Tale coppia viene fusa (*merged*) in un'unica nuova unità sintattica e aggiunta al vocabolario.
4. Il processo si ripete ricorsivamente fino al raggiungimento di una dimensione massima del vocabolario predefinita dal progettista (ad esempio, 32.000 o 128.000 token univoci).

Questo approccio permette ad ARIS di gestire con estrema efficienza il lessico dell'automazione industriale. Se una parola tecnica complessa o un acronimo specifico del controller Fanuc non è presente nel vocabolario principale, l'algoritmo BPE non fallisce, ma la decompone in sotto-unità morfologiche note (ad esempio, "troubleshooting" potrebbe essere segmentato nei token ["trouble", "shooting"]), preservandone la computabilità semantica.

2.2.2 Context Window e Iperparametri di Decodifica

Una volta convertito l'input in una sequenza di token, l'LLM alloca tali informazioni all'interno della propria **Context Window** (Finestra di Contesto). Da un punto di vista teorico, la dimensione della finestra è rigidamente vincolata dall'architettura dei Transformer, in cui il costo computazionale e l'occupazione di memoria RAM video (VRAM) del meccanismo di *Self-Attention* scalano in modo quadratico rispetto alla lunghezza della sequenza complessiva ($O(N^2)$). Sebbene i modelli recenti implementino ottimizzazioni algoritmiche per estendere tale limite, l'efficacia di recupero decade qualora si saturi la finestra, attivando derive informative causate dal sovraccarico computazionale.

Il processo con cui l'LLM estrae i token per comporre la risposta è intrinsecamente probabilistico e viene governato in fase di inferenza da specifici iperparametri matematici, la cui calibrazione è critica in un'applicazione di manutenzione industriale:

- **Temperatura (T):** Agisce direttamente sulla funzione *Softmax* applicata ai logit di output del modello. La distribuzione di probabilità riscaldata è definita come:

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2.2)$$

Dove z_i rappresenta il punteggio grezzo (*logit*) associato al token i -esimo. Quando la temperatura tende a zero ($T \rightarrow 0$), il modello applica una decodifica di tipo *Greedy Search*, azzerando la variabilità e selezionando rigorosamente solo il token a probabilità massima. In ambito industriale, impostare $T = 0.0$ è mandatorio per inibire la componente "creativa" del modello, forzando risposte deterministiche ed eliminando le allucinazioni sui codici operativi del robot Fanuc.

- **Top-P (Nucleus Sampling):** Introduce un vincolo dinamico sulla selezione dei token. Invece di considerare l'intero vocabolario, il modello ordina i candidati in ordine

decescente di probabilità e ne restringe la scelta al sottoinsieme più piccolo la cui somma cumulativa soddisfa la soglia $p \in [0, 1]$. Lavorando in sinergia con la temperatura, il Top-P taglia la "coda lunga" delle parole statisticamente improbabili, stabilizzando la coerenza logica della spiegazione tecnica.

2.2.3 Prompt Engineering, System Prompt e Agenti

L'interazione finale con il modello e l'effettiva integrazione del contesto recuperato dal Vector Database avvengono attraverso le tecniche di **Prompt Engineering**. All'interno di una pipeline RAG, il prompt non si configura come una semplice stringa di testo, ma come una struttura stratificata composta da tre livelli logici:

1. **System Prompt (Istruzioni di Sistema):** È lo strato gerarchicamente più elevato. Definisce il comportamento fisso, il ruolo e i vincoli operativi del modello (ad esempio: "Agisci come un ingegnere dell'automazione esperto in sistemi Fanuc. Rispondi basandoti esclusivamente sul contesto fornito e dichiara l'impossibilità di rispondere se l'informazione è assente").
2. **Contesto Iniettato (*Augmented Context*):** È la porzione di memoria dinamica riempita in tempo reale dai chunk di testo estratti dal Vector DB (le specifiche esatte del manuale **Fanuc R-30iB Mate**).
3. **User Prompt (Richiesta Utente):** La query o il codice di errore inserito dall'operatore sull'interfaccia di controllo.

L'evoluzione più recente dello stato dell'arte vede il superamento del RAG lineare a favore delle **Architetture ad Agenti AI** (*Agentic RAG*). Un Agente AI non si limita a eseguire una sequenza passiva di recupero e generazione, ma implementa un ciclo di ragionamento autonomo basato sul paradigma *ReAct* [9] (Reason and Act).

L'agente valuta la complessità della richiesta dell'utente e decide in autonomia, attraverso l'uso di *Tool* (funzioni esterne richiamabili programmaticamente), se interrogare il database vettoriale classico, attivare una ricerca ipertestuale sul grafo della conoscenza (*GraphRAG*), o concatenare più sotto-interrogazioni prima di formulare la diagnosi finale del guasto.

2.3 Scelta dei Modelli e Deployment

2.3.1 Cloud vs On-Premise: il trade-off tra API e Privacy Aziendale

La messa in produzione (*deployment*) di un assistente intelligente nel settore dell'automazione industriale impone un'attenta analisi di compromesso tra capacità computazionale, costi di gestione e sovranità del dato. Nello stato dell'arte moderno, questa scelta si polarizza tra due approcci infrastrutturali:

- **Infrastruttura Cloud (tramite API):** Consente di demandare il calcolo a server remoti, garantendo l'accesso a modelli di frontiera con capacità di ragionamento superiori e latenze ridotte, senza la necessità di investire in costoso hardware locale (GPU dedicate). Di contro, l'invio di prompt a server di terze parti espone l'azienda a potenziali rischi di conformità e violazione della riservatezza dei dati industriali.

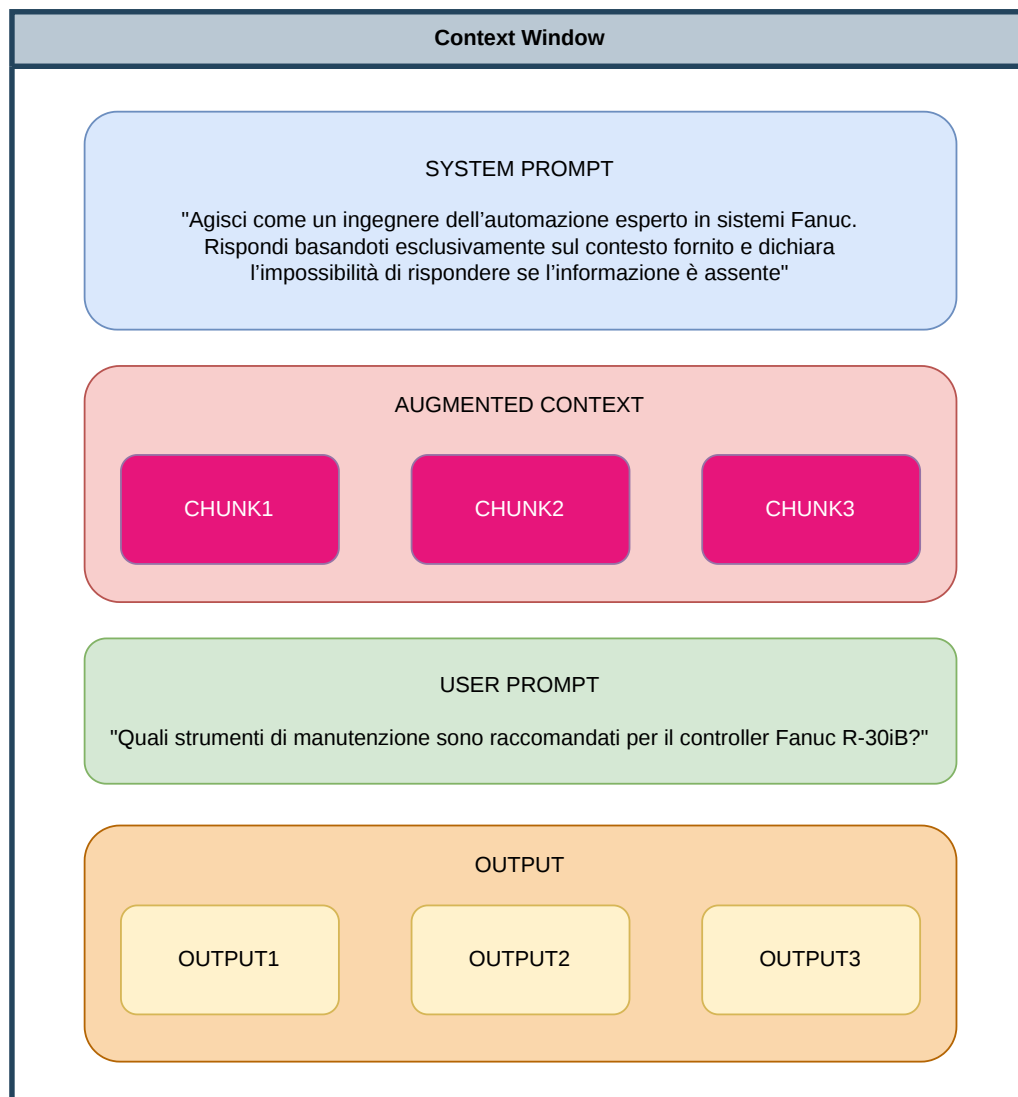


Figura 2.1: Schema di una Finestra di Contesto in un'architettura RAG

- **Infrastruttura On-Premise (Locale):** Prevede l'esecuzione dei modelli direttamente sui nodi computazionali della fabbrica. Questo garantisce una privacy aziendale assoluta (nessun dato abbandona la rete locale) e una continuità operativa indipendente dalla connessione Internet. Tuttavia, è rigidamente limitata dalla potenza di calcolo dell'hardware locale, costringendo spesso all'adozione di modelli più compatti o quantizzati.

Nel presente progetto si è scelto di implementare e testare entrambe le topologie, al fine di valutare empiricamente il *trade-off* prestazionale. Per l'ambiente cloud, la connessione è stata mediata dalla piattaforma **OpenRouter**. OpenRouter agisce come un aggregatore e router di API unificato, che ottimizza la pipeline instradando le richieste verso i provider cloud più efficienti in termini di costo e latenza, standardizzando l'interfaccia di chiamata. Per l'ambiente locale, la scelta è ricaduta su **LM Studio**, un framework *self-contained* che permette di istanziare un server di inferenza locale compatibile con le specifiche OpenAI sfruttando l'architettura `llama.cpp`, ideale per l'integrazione immediata nei flussi software.

2.3.2 Selezione dei Modelli Linguistici di Riferimento

La pipeline RAG sviluppata per il chatbot ARIS è stata validata integrando due specifici modelli linguistici di riferimento, scelti come massimi esponenti delle rispettive categorie d'infrastruttura:

1. **Cloud Baseline — GPT-4o-mini:** Modello di frontiera ottimizzato da OpenAI per compiti di ragionamento veloci ed economicamente sostenibili. Con una finestra di contesto estremamente capiente e un costo per milione di token ridotto, rappresenta il motore di generazione ideale in ambiente cloud, dimostrando spiccate capacità nella comprensione di istruzioni complesse presenti nel *system prompt*.
2. **Local Baseline — Phi-3.5 (Mini):** Modello open-weight sviluppato da Microsoft, caratterizzato da soli 3.8 miliardi di parametri. Phi-3.5 è stato selezionato per la sua straordinaria efficienza: nonostante le dimensioni ridotte (che ne consentono l'esecuzione fluida anche su macchine con risorse hardware e memoria RAM limitate), si attesta su livelli di benchmark elevatissimi nell'universo dei *Small Language Models* (SLM), surclassando architetture locali storicamente più pesanti in compiti di logica e sintesi documentale.

2.3.2.1 Evoluzioni dell'inferenza locale di frontiera — DwarfStar 4 (DS4)

Nel panorama dell'inferenza on-premise, i vincoli hardware legati alla larghezza di banda e alla memoria video rappresentano il collo di bottiglia principale per l'esecuzione di modelli di grandi dimensioni. In questo scenario, risulta di fondamentale rilevanza scientifica citare lo sviluppo di tecnologie d'avanguardia nate per scardinare tale limite, come il framework **DwarfStar 4 (DS4)** rilasciato dall'ingegnere Salvatore Sanfilippo (antirez, creatore di Redis⁶).

DS4 si configura come un motore di inferenza nativo in puro linguaggio C, privo di dipendenze esterne, specificamente ingegnerizzato per ottimizzare l'esecuzione locale di modelli quasi-frontier ad architettura Mixture-of-Experts (MoE⁷) — come *DeepSeek V4 Flash* — su

⁶Remote Dictionary Server, database residente in memoria di tipo key-value

⁷utilizzo di più reti neurali esperte (specializzate su un preciso compito)

workstation desktop. Sfruttando tecniche di quantizzazione asimmetrica a 2-bit (in cui i pesi degli esperti vengono compressi a 2-bit mentre l'attention e gli embedding sono preservati a 8-bit), DS4 dimostra come sia possibile far girare modelli da oltre 100 gigabyte riducendo drasticamente l'occupazione di memoria RAM senza compromettere le capacità di ragionamento e di chiamata ai tool. Sebbene nel presente lavoro l'hardware limitato abbia imposto l'uso del più leggero Phi-3.5, l'introduzione di soluzioni come DS4 traccia la rotta per il futuro dell'automazione industriale, dimostrando la fattibilità concreta di ospitare architetture di intelligenza artificiale di classe superiore interamente all'interno del perimetro fisico della fabbrica.

2.3.3 Osservabilità e Tracciamento delle Richieste via LangSmith

L'ottimizzazione di una pipeline RAG richiede uno strumento di telemetria e monitoraggio per ispezionare accuratamente il comportamento dei modelli durante le fasi di sviluppo e test. A tale scopo, l'intera architettura è stata interfacciata con **LangSmith**.

LangSmith fornisce uno strato di osservabilità (*observability layer*) essenziale che permette di mappare visivamente ogni singola transazione effettuata. Attraverso la sua interfaccia è possibile analizzare l'albero di esecuzione delle chiamate, visualizzando in tempo reale il testo esatto digitato dall'utente, i chunk specifici estratti dal Vector Database, il prompt finale iniettato e la risposta emessa dall'LLM. Questa granularità si rivela indispensabile in fase di debug industriale per identificare colli di bottiglia legati alla latenza di rete, monitorare il consumo e il costo dei token, e comprendere se un'eventuale risposta imprecisa sia da imputare a un fallimento del modulo di recupero (*Retrieval failure*) o a un errore di interpretazione del modello di generazione (*Generation failure*).

2.4 Metriche di valutazione

2.4.1 Metriche di valutazione e Framework di Testing

La validazione empirica dell'architettura del chatbot ARIS richiede l'implementazione di un rigido impianto metodologico di valutazione. Data la natura modulare del sistema, il protocollo di *benchmarking* è stato bipartito in due macro-fasi sequenziali e distinte: una valutazione preliminare focalizzata unicamente sulle performance del modulo di recupero (*Retrieval*) e una valutazione olistica orientata all'accuratezza della risposta finale generata dall'LLM.

2.4.1.1 Valutazione del Modulo di Retrieval

Questa prima sessione di test prescinde dalle capacità generative dell'LLM di base ed esamina esclusivamente la capacità del Vector Database di estrarre i frammenti informativi pertinenti estratti dalla manualistica del controller **Fanuc R-30iB Mate**. Le metriche matematiche e prestazionali adottate sono:

- **Hit Rate@k**: Misura la percentuale di casi in cui il chunk contenente la risposta esatta è presente all'interno dei primi k documenti recuperati dal database.

- **Precision@k:** Esprime la frazione di chunk effettivamente rilevanti rispetto al totale dei k chunk estratti, quantificando la densità di informazione utile somministrata al contesto.
- **Recall@k:** Indica la proporzione di informazioni rilevanti recuperate dal sistema rispetto al totale dei dati pertinenti teoricamente disponibili nel dataset di test.
- **Mean Reciprocal Rank (MRR):** Valuta l'ordinamento qualitativo dei chunk, calcolando la media dei reciproci delle posizioni (*rank*) in cui compare il primo frammento corretto. Un chunk corretto in prima posizione restituisce un punteggio pari a 1, in seconda 0.5, penalizzando i recuperi inefficienti.
- **Tempo medio di Retrieval (s):** Parametro prestazionale che quantifica la latenza computazionale del Vector DB durante le operazioni di ricerca semantica.

Al fine di calcolare tali metriche in modo oggettivo, ciascun chunk memorizzato mantiene un riferimento logico alla pagina del manuale d'origine. Nel calcolo statistico di Hit Rate, Precision e Recall è stata introdotta una **regola di tolleranza di una pagina** (± 1). Tale accorgimento geometrico si rende necessario poiché le procedure complesse del sistema Fanuc (es. le tabelle di ripristino allarmi) si estendono frequentemente a cavallo tra più pagine adiacenti; qualora l'algoritmo di *chunking* intercetti la porzione informativa corretta indicizzandola con la pagina immediatamente precedente o successiva rispetto alla *ground truth*, il recupero viene comunque computato come valido.

2.4.1.2 Valutazione della Generazione e Framework RAGAS

La seconda fase valida l'efficacia dell'interazione complessiva e la veridicità delle risposte fornite all'operatore sul campo. Oltre al monitoraggio dell'accuratezza generale (Hit Rate) e dei tempi di inferenza, questo livello di test sfrutta il framework **RAGAS** [10] (*Retrieval Augmented Generation Assessment*), uno standard d'eccellenza per la validazione automatizzata e *reference-free* dei sistemi RAG.

A differenza delle metriche classiche dell'NLP (come BLEU o ROUGE, inadeguate a misurare il significato concettuale), RAGAS isola la qualità della generazione scindendola in tre dimensioni core misurate in uno spettro $[0, 1]$ mediante l'ausilio di un LLM esterno operante in qualità di giudice (*LLM-as-a-Judge*):

- **Faithfulness (Fedeltà fattoriale):** Valuta la consistenza fattoriale della risposta rispetto al contesto iniettato. Il modello giudice scompone la risposta generata in singole asserzioni atomiche e verifica quante di esse siano logicamente derivabili dai chunk estratti dal manuale Fanuc, intercettando e penalizzando immediatamente qualsiasi allucinazione.
- **Answer Relevance (Pertinenza della Risposta):** Misura quanto la risposta generata aderisca alle reali necessità espresse dall'utente nella query, penalizzando output che, pur essendo fattualmente corretti, risultano ridondanti, incompleti o fuori fuoco rispetto all'anomalia descritta.
- **Context Precision (Precisione del Contesto):** Valuta la qualità dell'ordinamento dei chunk recuperati all'interno del prompt. Questa metrica calcola se i frammenti

informativi realmente pertinenti ed essenziali siano stati posizionati nelle primissime posizioni del contesto rispetto a chunk secondari o rumorosi. Un'elevata *Context Precision* garantisce che l'LLM di generazione riceva l'informazione critica immediatamente all'inizio della finestra, ottimizzando l'attenzione del modello ed evitando fenomeni di degradazione cognitiva.

- **Context Recall (Completezza del Recupero):** Analizza se il modulo di retrieval abbia estratto tutte le informazioni necessarie per formulare la risposta ideale, confrontando il contesto recuperato con la risposta di riferimento.

L'infrastruttura di test poggia su un dataset standardizzato composto da triplette composte da: domanda dell'operatore, risposta attesa e pagina di riferimento del manuale. Tali record sono stati preventivamente sintetizzati sfruttando le capacità cognitive di modelli di frontiera superiori rispetto a quelli target di ARIS, mentre la validazione finale in fase di inferenza è demandata a un modello LLM di classe *frontier* operante come *Judge*. I dettagli applicativi relativi alla generazione di tale dataset e alla configurazione del giudice verranno approfonditi all'interno del Capitolo 4.

Capitolo 3

Sviluppo ed Implementazione

In questo capitolo viene analizzata la messa a terra ingegneristica del sistema ARIS. Dopo una panoramica sintetica sull'infrastruttura preesistente di scomposizione e indicizzazione della manualistica, l'esposizione si concentra sul contributo originale di questo lavoro di tesi: l'ottimizzazione degli algoritmi di recupero avanzato, la strutturazione delle chiamate ai modelli linguistici sia in cloud sia on-premise, e lo sviluppo dell'interfaccia utente finale per gli operatori di linea.

3.1 Pipeline di Ingestion e Indicizzazione

3.1.1 Document Parsing

Nel corso dello sviluppo, la fase di *parsing* della manualistica tecnica si è rivelata un fattore determinante per l'accuratezza e la stabilità dell'intera pipeline di recupero. Estrarre informazioni strutturate da un documento tecnico complesso non si limita alla semplice lettura dei caratteri, ma richiede la preservazione dei legami logici tra testo, immagini e tabelle diagnostiche. Al fine di individuare la soluzione ottimale per la digitalizzazione dei manuali del controller **Fanuc R-30iB Mate**, sono stati implementati e posti a confronto sei differenti approcci metodologici:

- **Euristico:** Un algoritmo deterministico proprietario sviluppato ad-hoc, basato sulla conoscenza a priori della struttura impaginativa della documentazione Fanuc. Il sistema esegue un'analisi geometrica dello spazio della pagina, classificando i blocchi di testo in base alla dimensione dei font identificati per ogni sezione e alla loro posizione relativa. Sebbene sia progettato per ottenere riscontri di alto livello sul documento target, questo metodo non è generale e risulta fortemente sconsigliato in un'ottica di industrializzazione, poiché non è scalabile né facilmente replicabile su documenti di costruttori differenti senza una riscrittura del codice.
- **pdf4llm:** Un estrattore specialistico orientato ai Large Language Models, il cui obiettivo è convertire i dati interni del file PDF in una rappresentazione a marcatori in formato *Markdown*. Questa pipeline si concentra sulla conservazione della linearità del testo e sulla traduzione semantica delle tabelle in strutture testuali compatibili con i tokenizzatori degli LLM di frontiera.

- **Docling**: Un framework open-source di ultima generazione sviluppato per l'estrazione rapida e la destrutturazione di layout documentali imponenti. Utilizza euristiche avanzate per isolare i meta-dati, i paragrafi e le gerarchie interne, esportando il contenuto in formati standardizzati JSON o Markdown con un basso footprint computazionale.
- **LlamaParse**: Un servizio di parsing nativo e centralizzato basato su chiamate API, ingegnerizzato specificamente per la preparazione di dati destinati ad architetture RAG. Eccelle nell'identificazione e nel parsing isolato di tabelle complesse, diagrammi strutturati e contenuti nidificati all'interno di documenti con layout multi-colonna.
- **MinerU**: Uno strumento avanzato e integrato di *Content Extraction* che combina modelli di deep learning per il riconoscimento automatico dei blocchi di testo. È progettato per eliminare gli elementi di rumore (come intestazioni, piè di pagina e layout grafici geometrici) e convertire gli schemi tecnici in codice matematico o stringhe pulite.
- **Qwen (Vision-based)**: Un approccio radicalmente differente basato sull'impiego di un modello multimodale (*Vision-Language Model*) applicato all'analisi visiva delle pagine del manuale. Il modello non legge il codice binario del PDF, ma elabora la pagina come un'immagine *raster*¹, eseguendo un'operazione di comprensione contestuale e OCR integrato per ricostruire il testo.

I risultati sperimentali derivanti dall'applicazione di questi metodi sono illustrati nei grafici seguenti, i quali ne mappano rispettivamente l'accuratezza del recupero (espressa tramite Hit Rate@3 e Mean Reciprocal Rank) e l'efficienza temporale complessiva.

Come strategie di riferimento per condurre i test sono state selezionate le due architetture di retrieval più avanzate dello stato dell'arte, ovvero *Rerank* e *Graph*, al fine di valutare le prestazioni degli algoritmi di parsing sugli scenari applicativi migliori. Come si evince dalle metriche, le colonne relative a ciascun parser sono tra loro molto simili e stabili; questo comportamento è giustificato dal fatto che l'approccio GraphRAG non interviene per massimizzare in modo diretto le metriche di retrieval puro (come l'Hit Rate o l'MRR dei singoli chunk), ma è ingegnerizzato per supportare il modello in una fase più avanzata — quella di inferenza — arricchendo dinamicamente i chunk principali estratti con i nodi e le relazioni semantiche logicamente connesse nel grafo della conoscenza.

¹memorizzata come una griglia di pixel con un valore associato che indica il colore da assumere

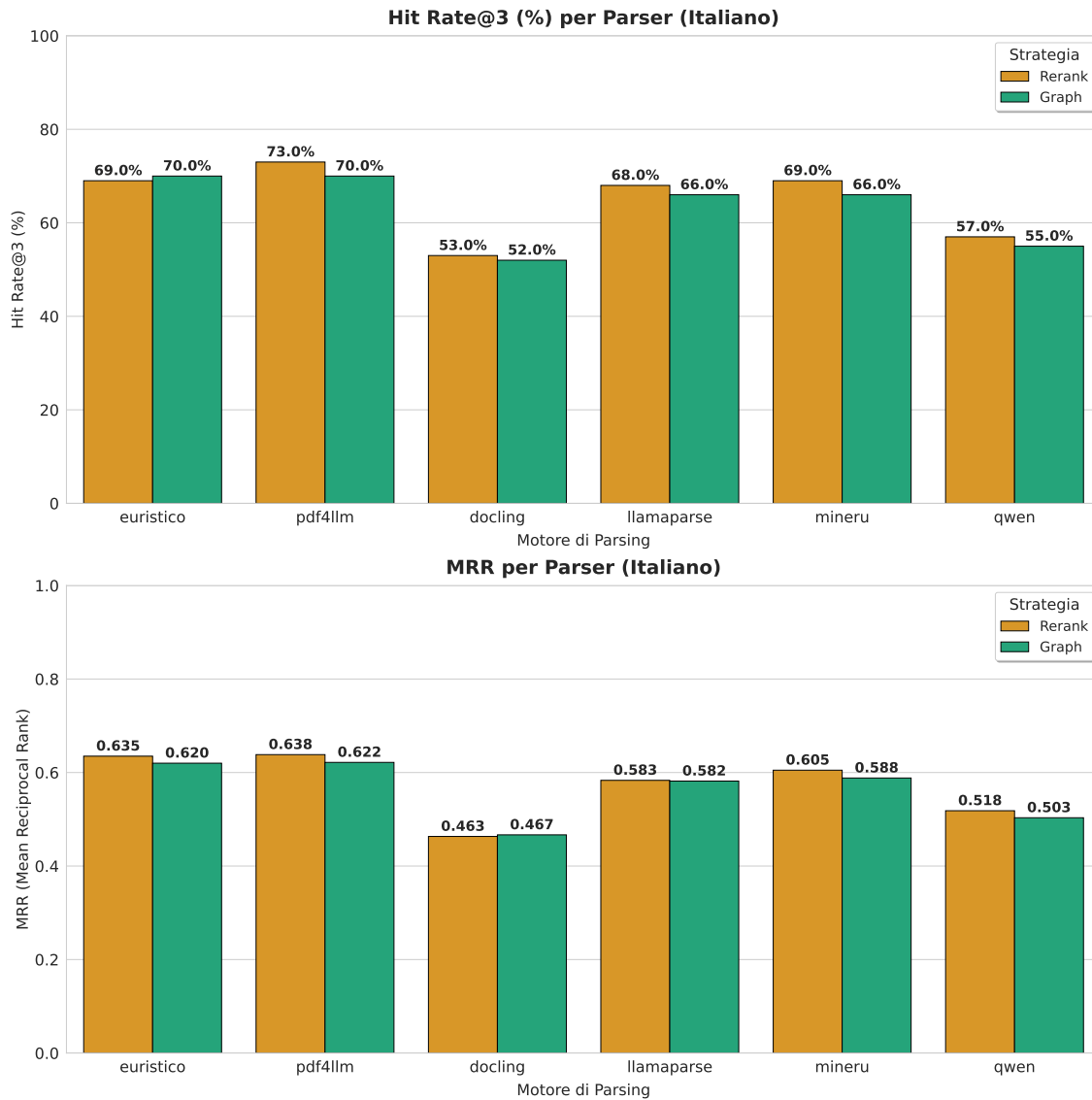


Figura 3.1: Istogramma Hit Rate ed MRR per metodo di parsing utilizzato

Per quanto riguarda l'accuratezza (Figura 3.1), il parser **pdf4llm** fa registrare le performance di picco in assoluto, raggiungendo un *Hit Rate@3* del 73.0% e un valore di *MRR* pari a 0.638. Questo risultato si attesta sensibilmente al di sopra dello stesso approccio **Euristico** (il quale si arresta a un Hit Rate del 69.0%), confermando l'efficacia di una scomposizione basata sulla formattazione logica del documento (*Markdown tagging*) rispetto a una rigida segmentazione basata puramente sulle coordinate geometriche e sulla dimensione dei font.

Al contrario, i metodi basati su modelli visivi o algoritmi complessi di destrutturazione, quali **Qwen** e **Docling**, mostrano una marcata contrazione delle prestazioni, registrando i valori minimi del lotto con un Hit Rate che cala fino al 52.0% nel caso di Docling. Questo andamento evidenzia che, operando su file digitali nativi ad alta definizione, l'intermediazione di layer di parsing complessi o l'astrazione tramite modelli di visione computazionale rischia di introdurre rumore algoritmico latente, frammentando la continuità informativa prima del caricamento e dell'indicizzazione nel database vettoriale.

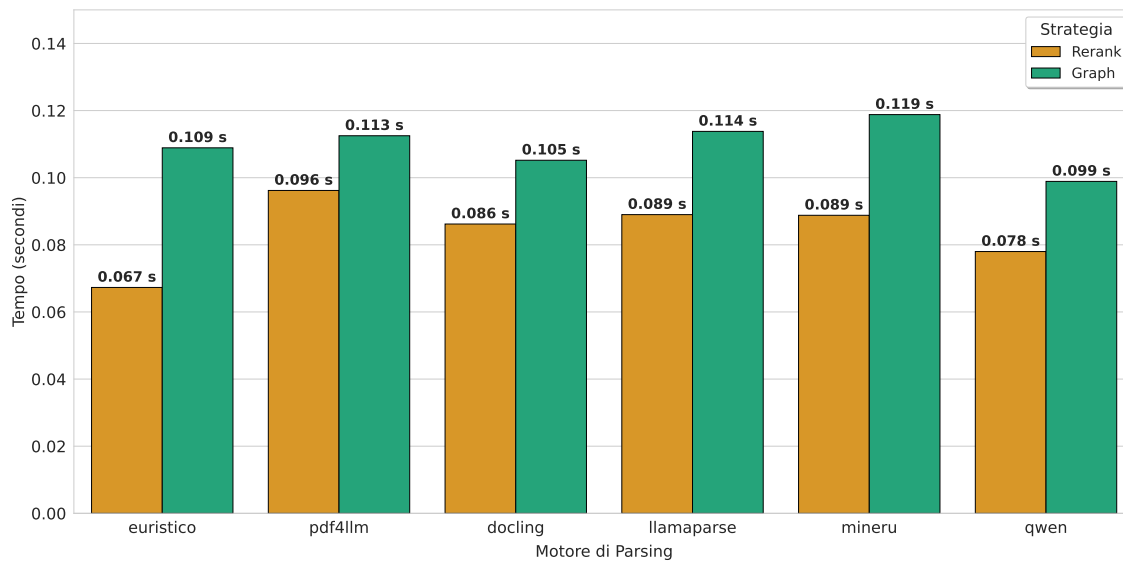


Figura 3.2: *Analisi comparativa della latenza media di retrieval (espressa in secondi) per metodo di parsing utilizzato*

In merito all'efficienza temporale (Figura 3.2), espressa tramite il tempo medio in secondi richiesto per completare l'operazione complessiva di retrieval, l'andamento si inverte. I motori che hanno registrato la minore accuratezza semantica si dimostrano, di contro, i più rapidi nell'elaborazione: l'approccio **Euristico** e il modello visivo **Qwen** registrano infatti le latenze più basse in assoluto (rispettivamente 0.067 s e 0.078 s), offrendo una spiccata reattività computazionale a runtime.

Il parser **pdf4llm** richiede un tempo medio leggermente superiore alla media del lotto (pari a 0.096 s con strategia Rerank), ma tale scostamento nell'ordine dei millisecondi risulta del tutto trascurabile ed impercettibile per un operatore o un manutentore durante una reale sessione di chat interattiva sul campo.

In conclusione, **pdf4llm** si configura come il miglior compromesso ingegneristico per lo sviluppo del sistema ARIS: a fronte di un costo in latenza ampiamente tollerabile e in linea con i requisiti software in tempo reale, garantisce il più alto livello di accuratezza e precisione nell'ancoraggio delle procedure di manutenzione estratte dal manuale del controller Fanuc.

3.1.2 Strategie di Chunking e Generazione degli Embedding

Una volta estratto e ripulito il testo semantico dai manuali tecnici del controller Fanuc, si rende necessario eseguire la fase di *Chunking* (segmentazione). Questa operazione è di fondamentale importanza: poiché la conoscenza viene immagazzinata all'interno di un database vettoriale, la dimensione di ciascun frammento condiziona direttamente sia la precisione del recupero semantico (*Retrieval*) sia l'efficienza della memoria della *context window*.

All'interno dell'architettura ARIS, la scomposizione del testo continuo è stata ingegnerizzata a dimensione fissa basandosi sul conteggio effettivo dei token, differenziando l'algoritmo a seconda della natura del modello target (Cloud o On-Premise). Per preservare la coerenza logica ed evitare che stringhe di testo o codici di errore (come gli identificativi diagnostici SRVO o SYST) vengano troncati a metà tra due blocchi adiacenti, è stato configurato un parametro di *Overlap* [7] (sovrapposizione) dinamico, proporzionale alla dimensione del chunk.

L'implementazione software di questa logica di partizionamento si appoggia alle classi specializzate del framework LangChain². La porzione saliente del codice sorgente deputata all'inizializzazione dei motori di segmentazione ricorsiva è riportata nel Listing 3.1.

Listing 3.1: Logica di inizializzazione del RecursiveCharacterTextSplitter per modelli Cloud e Locali.

```

1  # Selezione dello splitter in base alla natura del modello ed
    embedding target
2  if tipo_modello == "cloud":
3      # Utilizza il tokenizzatore tiktoken nativo di OpenAI
4      text_splitter = RecursiveCharacterTextSplitter.
        from_tiktoken_encoder(
5          model_name=nome_modello,
6          chunk_size=chunk_size,
7          chunk_overlap=chunk_overlap,
8          separators=["\n\n", "\n", ".", " ", ""]
9      )
10 elif tipo_modello == "locale":
11     # Carica il tokenizzatore da HuggingFace (es. BAAI/bge-m3)
12     tokenizer = AutoTokenizer.from_pretrained(nome_modello)
13
14     text_splitter = RecursiveCharacterTextSplitter.
        from_huggingface_tokenizer(
15         tokenizer=tokenizer,
16         chunk_size=chunk_size,
17         chunk_overlap=chunk_overlap,
18         separators=["\n\n", "\n", ".", " ", ""]
19     )

```

Come si evince dal codice, il testo viene analizzato applicando un ordine gerarchico di separatori (["\n\n", "\n", ".", " ", ""]). Il sistema tenta inizialmente di dividere il file in corrispondenza dei doppi a capo (interruzioni di paragrafo); qualora la porzione ecceda il valore di `chunk_size`, l'algoritmo scende progressivamente ai singoli a capo, ai punti e infine agli spazi bianchi, massimizzando la coerenza sintattica del blocco risultante.

²Framework open source utilizzato per applicazioni che prevedono l'utilizzo di LLM

3.1.2.1 Analisi di Sensibilità al Chunk Size

Al fine di determinare la configurazione geometrica ottimale per la base di conoscenza, è stata condotta un'analisi di sensibilità testando tre diverse lunghezze di chunking (300, 700 e 1000 token) a parità di condizioni di estrazione (eseguita con il parser di riferimento **pdf4llm**) e modello di embedding locale (**BAAI/bge-m3**). I risultati empirici per le due macro-strategie di recupero analizzate sono sintetizzati nelle tabelle seguenti.

Database / Configurazione	Hit Rate@3 (%)	Precision@3 (%)	Recall@3 (%)	MRR	Tempo Medio (s)
chroma_pdf4llm_locale_300	69.0	36.67	69.0	0.6067	0.0298
chroma_pdf4llm_locale_700	71.0	36.67	71.0	0.6417	0.0220
chroma_pdf4llm_locale_1000	71.0	35.67	71.0	0.6267	0.0241

Tabella 3.1: *Analisi di sensibilità del Chunk Size applicata al Retrieval Vettoriale.*

Database / Configurazione	Hit Rate@3 (%)	Precision@3 (%)	Recall@3 (%)	MRR	Tempo Medio (s)
chroma_pdf4llm_locale_300	68.0	37.00	68.0	0.6017	0.0266
chroma_pdf4llm_locale_700	71.0	36.67	71.0	0.6167	0.0232
chroma_pdf4llm_locale_1000	70.0	36.00	70.0	0.6100	0.0263

Tabella 3.2: *Analisi di sensibilità del Chunk Size applicata al Retrieval Ibrido.*

L'ispezione comparativa dei dati raccolti nelle Tabella 3.1 e 3.2 evidenzia un comportamento ad andamento parabolico delle metriche di accuratezza rispetto alla dimensione del blocco testuale.

Una dimensione ristretta di chunking (300 token) si dimostra parzialmente inefficiente, registrando i valori più bassi di *Hit Rate* e *MRR* in entrambe le configurazioni. Questo fenomeno è legato alla frammentazione eccessiva: stringhe informative complesse o lunghe tabelle d'allarme Fanuc vengono separate in blocchi distinti, disperdendo la correlazione semantica e riducendo la capacità del database vettoriale di tracciare la risposta esatta.

Di contro, espandere il chunk size fino a 1000 token non genera un incremento prestazionale; al contrario, nel caso del retrieval ibrido, si osserva una leggera flessione dell'*Hit Rate* (70.0%) e un incremento dei tempi medi di computazione. Un chunk eccessivamente ampio introduce infatti informazioni di contorno estranee alla richiesta specifica (*rumore di contesto*), diluendo la densità semantica del vettore di embedding.

I dati indicano in modo concorde che la configurazione a **700 token** costituisce il punto di ottimo ingegneristico per il sistema ARIS. In tale impostazione si registrano i massimi valori di accuratezza ed ordinamento (*Hit Rate* al 71.0% e *MRR* a 0.6417 nel retrieval puro), associati al tempo di risposta più basso in assoluto (0.022 s). Questa dimensione garantisce il perfetto equilibrio tra la conservazione dell'integrità concettuale del paragrafo tecnico e la massima reattività a runtime del Vector DB.

3.1.3 Archiviazione e Struttura del Database Vettoriale

Una volta convertiti i blocchi di testo in rappresentazioni matematiche dense attraverso i modelli di embedding, l'architettura ARIS necessita di un'infrastruttura di persistenza ottimizzata per l'indicizzazione e la ricerca topologica ad alta dimensione. A tale scopo è stato adottato **ChromaDB**, un database vettoriale open-source strutturato nativamente per interfacciarsi con i framework di orchestrazione LLM ed eseguire computazioni di vicinanza geometrica in tempi millisecondari.

La logica di popolamento del database vettoriale, implementata nello script software del progetto, prevede una netta separazione tra il vettore semantico, il contenuto testuale puro e le informazioni strutturate di contorno. Ogni record estratto dai file JSON di scomposizione viene convertito in un oggetto `Document` di LangChain secondo un paradigma a due livelli:

1. **Page Content:** Corrisponde alla stringa di testo estratta dal manuale Fanuc (il corpo del paragrafo o la riga della tabella). Questo elemento viene processato dal modello di embedding per calcolarne le coordinate spaziali.
2. **Metadata:** Un dizionario di variabili di controllo (quali `chunk_id`, `file_name`, `page`, `section`, `has_table` e `has_alarm_code`) che viene iniettato nel database parallelamente al vettore. Questi metadati non influiscono sul calcolo della distanza semantica, ma risultano essenziali nelle fasi successive per applicare filtri deterministici (ad esempio, circoscrivere la ricerca a una specifica sezione o risalire alla pagina esatta del manuale da mostrare all'operatore).

L'inizializzazione del database e la generazione differenziata dei vettori in base all'ambiente di esecuzione selezionato (Cloud o On-Premise) è governata dalla porzione di codice sorgente riportata nel Listing 3.2.

Listing 3.2: Logica di inizializzazione dei modelli di embedding e popolamento di ChromaDB.

```

1 # Selezione dell'hardware e del modello di embedding
2 if args.env == "locale":
3     device = "cuda" if torch.cuda.is_available() else "cpu"
4     # Inizializzazione del modello locale
5     embeddings = HuggingFaceEmbeddings(
6         model_name="BAAI/bge-m3",
7         model_kwargs={'device': device},
8         encode_kwargs={'normalize_embeddings': True}
9     )
10    persist_dir = os.path.join("vector_db", f"chroma_{args.metodo}_
        _locale_{args.chunk_size}")
11 else:
12     # Inizializzazione del modello cloud tramite OpenRouter
13     embeddings = OpenAIEmbeddings(
14         model="text-embedding-3-small",
15         openai_api_key=os.getenv("OPENAI_API_KEY"),
16         openai_api_base="https://openrouter.ai/api/v1"
17     )
18    persist_dir = os.path.join("vector_db", f"chroma_{args.metodo}_
        _cloud_{args.chunk_size}")
19
20 # Generazione dei vettori e scrittura persistente su disco
21 db = Chroma.from_documents(
22     documents=docs,
23     embedding=embeddings,
24     persist_directory=persist_dir
25 )

```

Come si evince dall'architettura dello script, qualora venga selezionato l'ambiente locale, il sistema interroga la disponibilità di core di calcolo su GPU mediante l'allocazione `cuda` di PyTorch, ottimizzando la velocità di embedding del modello BAAI/bge-m3. Nel caso di ambiente cloud, le chiamate vengono instradate verso l'endpoint standardizzato di OpenRouter per l'attivazione del modello `text-embedding-3-small`.

3.2 Strategie di Retrieval Avanzato

All'interno dell'architettura del sistema ARIS sono state implementate e poste a confronto quattro diverse strategie di *Retrieval*, disposte in ordine di complessità algoritmica e computazionale crescente. Ciascun livello incrementale introduce moduli logici aggiuntivi progettati per massimizzare la precisione e la densità informativa dei contesti iniettati nel prompt, bilanciando l'accuratezza diagnostica con la latenza di esecuzione della pipeline a runtime.

3.2.1 Retrieval Semantico Singolo (Dense Retrieval)

Il paradigma di **Retrieval Semantico Singolo**, noto nello stato dell'arte come *Dense Retrieval*, rappresenta l'approccio tradizionale e strutturalmente più lineare. In questa configurazione, l'interrogazione immessa dall'operatore viene interamente mappata in un unico vettore denso ad alta dimensionalità sfruttando il modello di embedding attivo. Successivamente, il sistema esegue una ricerca geometrica sui vettori memorizzati nel Vector DB mediante il calcolo dei vicini più prossimi (*k*-Nearest Neighbors, *k*-NN), utilizzando la similitudine coseno per estrarre i primi *k* chunk a massima affinità concettuale.

Sebbene questo modello eccella nell'intercettare il contesto astratto di una richiesta (ad esempio, associando la query "surriscaldamento del motore" ai paragrafi operativi relativi alle anomalie termiche), esso presenta un limite intrinseco: la proiezione geometrica densa tende a diluire il peso delle singole parole chiave e dei codici alfanumerici specifici, i quali costituiscono l'ossatura fondamentale della diagnostica industriale.

3.2.2 Retrieval Ibrido (Dense + Sparse/BM25)

Per ovviare alla perdita di granularità lessicale tipica dei vettori densi, si rende necessario accoppiare la ricerca semantica con una ricerca testuale esplicita a codifica sparsa (*Sparse Retrieval*). I manuali tecnici del controller **Fanuc R-30iB Mate** sono densamente popolati da stringhe identificative univoche — quali codici di errore (es. `SRV0-062`), sigle di schede elettroniche o nomi di registri di sistema — che richiedono una corrispondenza lessicale esatta (*keyword matching*).

L'architettura ARIS implementa pertanto una strategia combinata di tipo **Ensemble**, fondendo un modulo di recupero denso (ChromaDB) con un modulo di recupero sparso basato sull'algoritmo **BM25** (*Best Matching 25*), assegnando a ciascun motore un peso paritetico del 50%.

3.2.2.1 La transizione teorica da TF-IDF a BM25

A differenza del classico modello statistico **TF-IDF** (*Term Frequency - Inverse Document Frequency*), che calcola il peso di un termine in modo puramente lineare rispetto alla sua

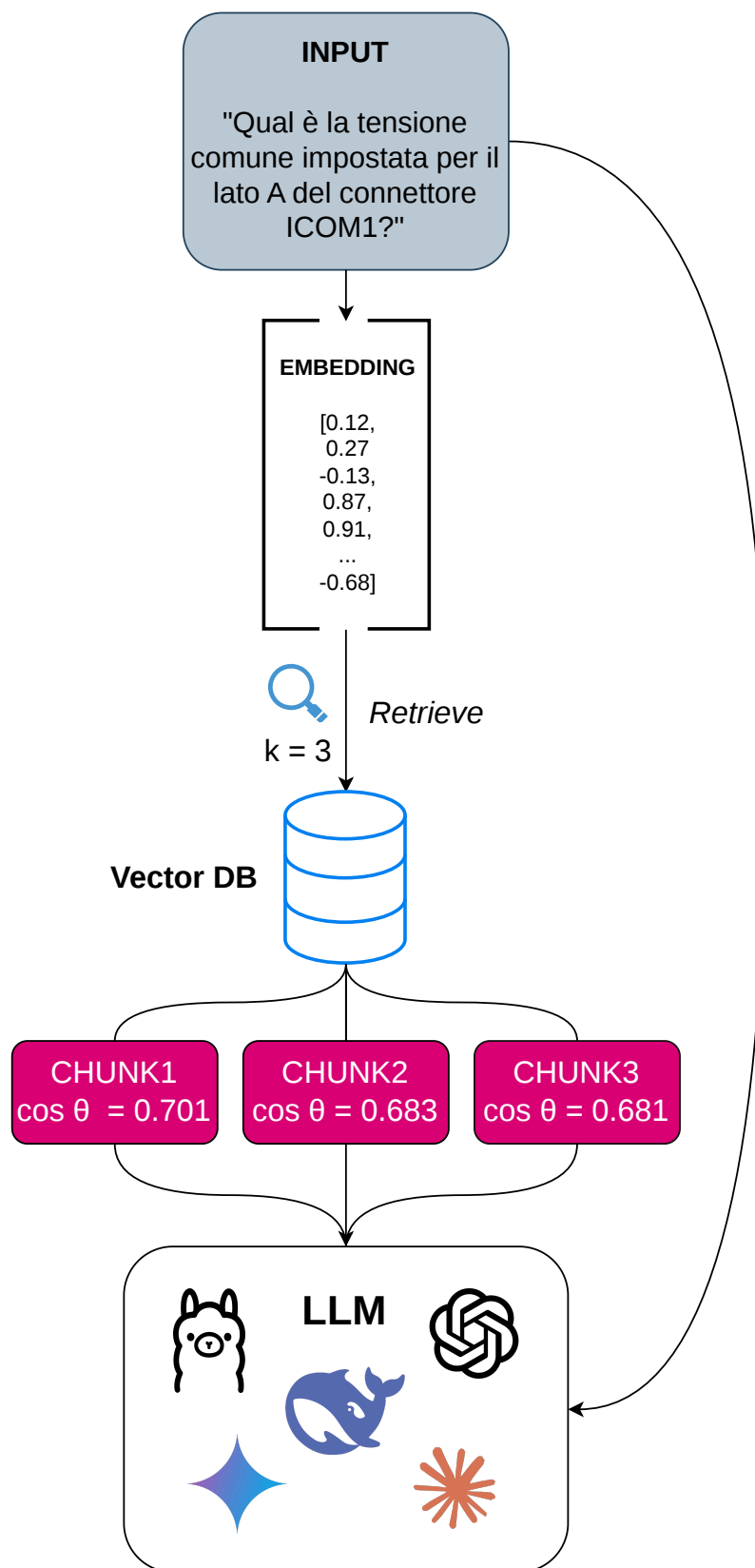


Figura 3.3: Schema pipeline Naive RAG

frequenza nel documento, l'algoritmo probabilistico **BM25** [11] introduce due correzioni non lineari fondamentali per la manualistica tecnica:

1. **Saturazione della Term Frequency:** In TF-IDF, se una parola chiave compare cento volte in un chunk, il suo score cresce indefinitamente in modo proporzionale. In BM25, la frequenza del termine viene smorzata in modo asintotico attraverso un parametro non negativo k_1 (tipicamente compreso tra 1.2 e 2.0). Raggiunta la soglia di saturazione, l'ulteriore presenza del termine non incrementa il punteggio, evitando che chunk ripetitivi oscurino paragrafi più densi e mirati.
2. **Normalizzazione della lunghezza del documento (Document Length Normalization):** I manuali Fanuc presentano chunk strutturalmente disomogenei (procedure sintetiche alternate ad ampie tabelle diagnostiche). BM25 normalizza il punteggio in base alla lunghezza del chunk esaminato rispetto alla lunghezza media di tutti i chunk del database (*avgdl*), modulata da un parametro di scaling $b \in [0, 1]$. Se $b = 0.75$, il sistema penalizza i chunk eccessivamente lunghi in cui la parola chiave compare per puro caso in mezzo a informazioni non pertinenti, premiando invece i frammenti brevi e focalizzati.

3.2.2.2 Il vincolo della lingua e il Translation Layer

Un limite strutturale della ricerca sparsa (BM25) risiede nella sua rigidità lessicale: se la query dell'operatore è formulata in lingua italiana (es. "asse surriscaldato") e la documentazione tecnica indicizzata è redatta in lingua inglese ("*axis overheated*"), la corrispondenza delle stringhe fallisce simmetricamente (*score* = 0.0).

Per superare questo vincolo senza rinunciare ai vantaggi del keyword matching sui codici d'errore, è stato sviluppato un **Layer di Traduzione deterministico** inserito a monte del recupero. Sfruttando un'infrastruttura di *Query Routing* gestita tramite un modello linguistico leggero operante a temperatura zero, il sistema intercetta la richiesta in input e, qualora rilevi l'uso della lingua italiana, ne esegue la traduzione automatica in lingua inglese prima di interrogare l'indice BM25.

3.2.2.3 Implementazione dell'Ensemble

Una volta normalizzata e tradotta la stringa di ricerca, il sistema istanzia e concatena i due canali di recupero paralleli. La logica di programmazione scritta per l'estrazione e l'unificazione bilanciata dei vettori densi e sparsi si appoggia all'oggetto **EnsembleRetriever** di LangChain.

Dal punto di vista algoritmico, fondere i risultati provenienti da un modello denso (ChromaDB) e uno sparso (BM25) solleva un problema matematico non banale: i due motori generano punteggi operanti su scale di magnitudo e distribuzioni di probabilità completamente eterogenee (la similitudine coseno è confinata nell'intervallo $[0, 1]$, mentre lo score BM25 è un valore teoricamente illimitato superiormente). Per aggirare questo limite e uniformare i risultati senza dover normalizzare arbitrariamente i punteggi grezzi, l'oggetto **EnsembleRetriever** implementa l'algoritmo di **Reciprocal Rank Fusion (RRF)**.

L'RRF è una tecnica di fusione indipendente dai punteggi nativi, basata esclusivamente sulla posizione (*rank*) che un determinato chunk occupa all'interno delle singole classifiche di output. Dato un insieme di documenti D e un set di sistemi di recupero R (nel nostro

caso, $R = \{BM25, Chroma\}$), il punteggio RRF associato a un singolo documento $d \in D$ è calcolato mediante la seguente formulazione matematica:

$$RRF_Score(d \in D) = \sum_{r \in R} \frac{w_r}{k + r(d)} \quad (3.1)$$

Dove:

- $r(d)$ rappresenta la posizione in classifica (*rank*) del documento d all'interno del sistema di recupero r . Se il chunk è il primo frammento estratto da ChromaDB, allora $r(d) = 1$.
- k è una costante di penalizzazione o costante di *smoothing* (lo standard algoritmico, adottato di default anche da LangChain, imposta $k = 60$). Questa costante serve a mitigare l'impatto di chunk che si posizionano molto in alto in una sola classifica per puro caso, stabilizzando l'influenza dei primi segmenti rispetto a quelli di coda.
- w_r rappresenta il peso ponderato assegnato al sistema di recupero r . Come definito a livello di configurazione nel sistema ARIS, viene applicato un peso paritetico del 50% per ciascun canale, impostando $w_{BM25} = 0.5$ e $w_{Chroma} = 0.5$.

Durante l'esecuzione, l'algoritmo esamina i chunk estratti da entrambi i canali, calcola il rispettivo RRF_Score e riordina i documenti in un'unica graduatoria finale in ordine decrescente di punteggio. I primi k documenti risultanti da questa fusione d'alto livello vengono infine veicolati nel prompt.

La messa a terra di questa architettura d'insieme all'interno del codice sorgente di ARIS è riportata nel Listing 3.3.

Listing 3.3: Costruzione del Retriever Ibrido Ensemble con fusione al 50% tra BM25 e ChromaDB.

```

1 def build_hybrid_retriever(db: Chroma, k: int = 3) ->
  EnsembleRetriever:
2     # 1. Inizializzazione del Retriever denso (Vector Search)
3     chroma_retriever = db.as_retriever(search_kwargs={"k": k})
4
5     # 2. Estrazione dei documenti per l'inizializzazione locale dell'
      indice BM25
6     all_data = db.get()
7     if not all_data or not all_data.get("documents"):
8         raise RuntimeError("Nessun documento trovato nel database
          vettoriale.")
9
10    docs = [
11        Document(page_content=text, metadata=meta)
12        for text, meta in zip(all_data["documents"], all_data["
          metadatas"])
13        if text
14    ]
15    print(f"BM25 inizializzato su {len(docs)} chunk...")
16    bm25_retriever = BM25Retriever.from_documents(docs)
17    bm25_retriever.k = k
18

```

```

19 # 3. Ensemble: fusione lineare ponderata (50% keyword + 50%
    semantic)
20 ensemble = EnsembleRetriever(
21     retrievers=[bm25_retriever, chroma_retriever],
22     weights=[0.5, 0.5]
23 )
24 return ensemble

```

3.2.3 Applicazione del Re-ranking

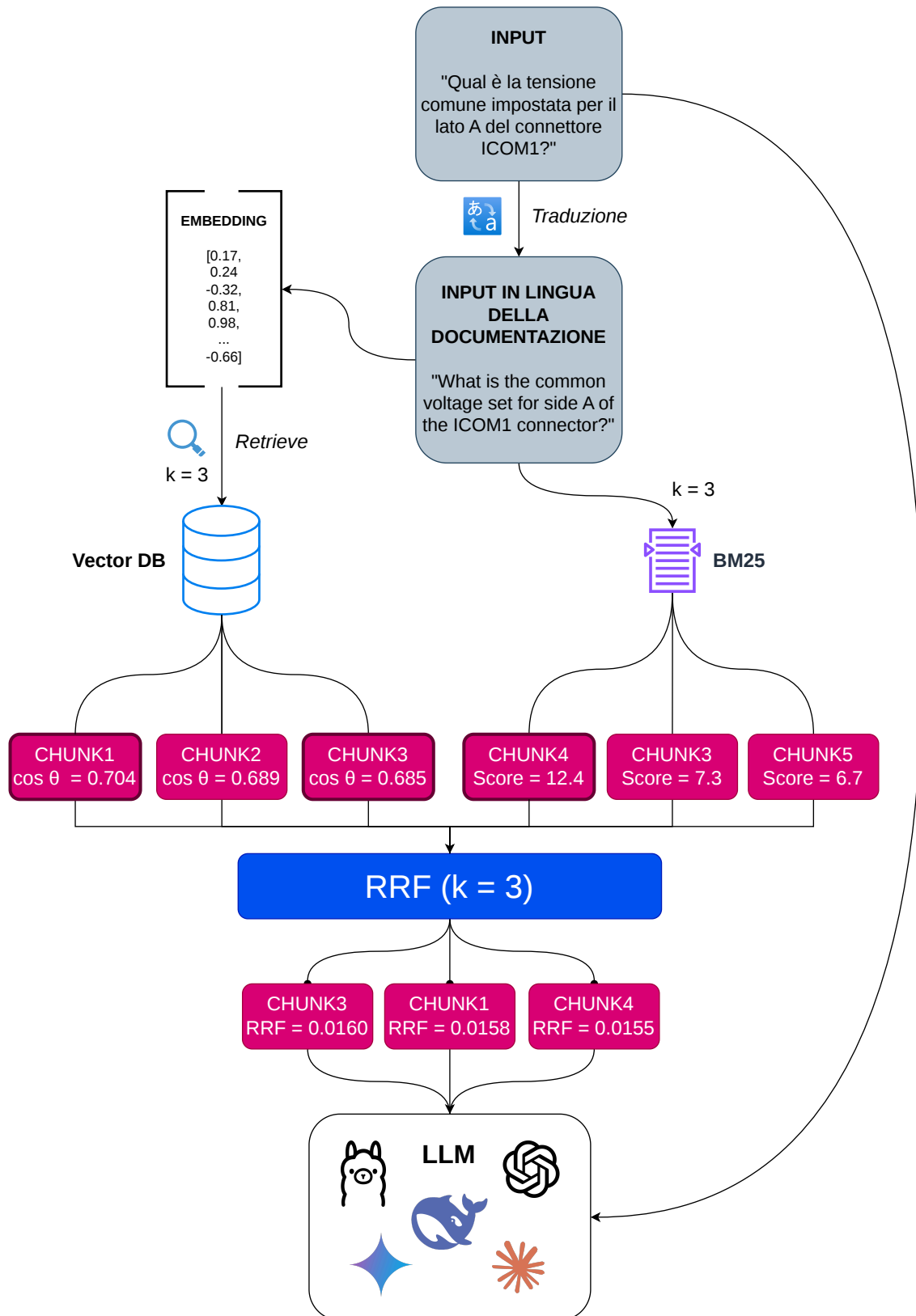
L'introduzione del recupero ibrido (vettoriale e lessicale) permette di intercettare in modo efficace sia i concetti astratti sia i codici d'errore espliciti. Tuttavia, per garantire che l'LLM operi al massimo delle sue capacità conoscitive, non è sufficiente che i chunk rilevanti siano presenti all'interno della finestra di contesto, ma è fondamentale che essi siano posizionati nelle primissime posizioni del prompt. Questo requisito risponde alla necessità di massimizzare la *Context Precision* ed evitare i cali di attenzione dei modelli legati al fenomeno del *Lost-in-the-Middle*.

Per risolvere questo problema, l'architettura ARIS introduce una strategia di recupero a due stadi (**Two-Stage Retrieval**) implementando un modulo di **Re-ranking** basato su una rete neurale di tipo **Cross-Encoder** [11].

3.2.3.1 Il divario teorico tra Bi-Encoder e Cross-Encoder

Dal punto di vista della teoria dell'*Information Retrieval*, la scomposizione delle due fasi è motivata da un preciso *trade-off* tra scalabilità computazionale e precisione semantica:

- **Primo Stadio (Bi-Encoder):** Modelli come BAAI/bge-m3 o text-embedding-3-small sono detti *Bi-Encoder* poiché elaborano la query dell'utente e i chunk documentali in modo totalmente indipendente. I vettori dei chunk vengono calcolati *una tantum* in modalità offline e stoccati nel database. A runtime, il sistema deve calcolare l'embedding della sola query ed eseguire un prodotto scalare rapido. Questa architettura è estremamente veloce e scala su milioni di documenti, ma non permette un'interazione profonda e bidirezionale tra i termini della domanda e quelli del testo.
- **Secondo Stadio (Cross-Encoder):** Il modello di Re-ranking, per il quale è stata selezionata l'architettura multilingua cross-encoder/mmarco-mMiniLMv2-L12-H384-v1, opera in modo radicalmente opposto. Essa riceve in input simultaneamente la query e il singolo chunk di testo, processandoli all'interno di un unico layer di auto-attenzione del Transformer. Questo permette al modello di calcolare un'attenzione incrociata termine-per-termine, generando un punteggio di pertinenza neurale diretta estremamente preciso. Poiché l'elaborazione simultanea di query e documento ha un costo computazionale elevato ($O(N^2)$), l'applicazione di un Cross-Encoder è impraticabile sull'intero database vettoriale, ma diventa altamente efficiente se ristretta a un pool circoscritto di documenti candidati estratti dal primo stadio.

**Figura 3.4:** Schema pipeline Hybrid RAG

3.2.3.2 Ingegnerizzazione del codice e gestione dei parametri k

La pipeline di Re-ranking implementata nel sistema ARIS opera secondo una precisa dinamica di espansione e contrazione del volume dei dati, governata dalla classe custom

`HybridRerankRetriever`. La gestione dei parametri di recupero è così strutturata:

1. **Generazione del pool allargato** ($k_{base} = 6$): In prima istanza, la query viene inoltrata all'istanza di `EnsembleRetriever`. Per massimizzare la probabilità di includere il contesto corretto nel primo passaggio, il parametro di recupero viene forzato a un livello più alto rispetto allo standard, impostando $k_{base} = 6$ per ciascun canale. Il sistema interroga parallelamente l'indice BM25 e ChromaDB, raccogliendo un pool iniziale potenziale di 12 chunk.
2. **Deduplicazione logica**: Lo script esegue un ciclo di hashing sul contenuto testuale grezzo (`page_content`) per rimuovere i chunk identici estratti contemporaneamente da entrambi i motori del primo stadio, stabilizzando la cardinalità del pool di candidati.
3. **Inferenza del Cross-Encoder**: Ciascuna coppia [Query, Chunk] viene somministrata alla rete neurale per calcolarne lo score di pertinenza. L'array risultante viene ordinato in graduatoria decrescente.
4. **Saturazione del contesto** ($k_{final} = 3$): Il retriever applica un filtro di soltimento (`top_n`) impostando $k_{final} = 3$. Solo i primi tre chunk a massimo punteggio neurale vengono promossi e veicolati all'LLM di generazione, scartando i restanti elementi del pool.

La traduzione software di questo algoritmo all'interno del backend del progetto è documentata dal codice sorgente Python riportato nel Listing 3.4.

Listing 3.4: Implementazione del modulo personalizzato `HybridRerankRetriever` per il Re-ranking neurale a due stadi.

```

1 class HybridRerankRetriever:
2     """Retriever Custom che unisce BM25 + Vector Search (k=12) e
3     applica Cross-Encoder Reranking (top_n=3)."""
4     def __init__(self, ensemble_retriever, db=None, model_name: str =
5         "cross-encoder/mmarco-mMiniLMv2-L12-H384-v1", top_n: int = 3,
6         debug: bool = False):
7         device = "cuda" if torch.cuda.is_available() else "cpu"
8         self.ensemble_retriever = ensemble_retriever
9         self.db = db
10        self.model = CrossEncoder(model_name, device=device)
11        self.top_n = top_n
12        self.debug = debug
13
14    def invoke(self, query: str) -> list:
15        # 1. Recupera i chunk candidati espansi dal modulo ibrido
16        #     base (k=6 per ciascuno)
17        docs = self.ensemble_retriever.invoke(query)
18        if not docs:
19            return []

```

```

16
17     # Rimozione deterministica dei duplicati basati sul contenuto
        testuale
18     seen = set()
19     unique_docs = []
20     for doc in docs:
21         if doc.page_content not in seen:
22             seen.add(doc.page_content)
23             unique_docs.append(doc)
24
25     # 2. Esecuzione del Re-ranking neurale mediante Cross-Encoder
26     pairs = [[query, doc.page_content] for doc in unique_docs]
27     scores = self.model.predict(pairs)
28
29     # Associazione dei punteggi ai rispettivi oggetti Document e
        ordinamento decrescente
30     scored_docs = sorted(zip(scores, unique_docs), key=lambda x:
        x[0], reverse=True)
31
32     # Estrazione e restituzione dei soli primi top_n (k_final =
        3) frammenti
33     reranked_docs = [doc for _, doc in scored_docs[:self.top_n]]
34     return reranked_docs

```

Attraverso questo impianto, lo script ottimizza i flussi di calcolo: la memoria RAM e i core GPU (selezionati dinamicamente tramite allocazione `cuda` ove disponibili) vengono sollecitati per analizzare un numero ridotto di stringhe ($max = 12$). Questo garantisce che la latenza computazionale complessiva della pipeline rimanga confinata nell'ordine delle frazioni di secondo, fornendo al contempo all'operatore un'interfaccia iper-ottimizzata dal punto di vista della precisione documentale.

3.2.4 Integrazione e Architettura GraphRAG

La quarta e più avanzata strategia di recupero implementata nel sistema ARIS introduce il paradigma del **GraphRAG** (*Graph-Expanded Re-ranked Retrieval*). Sebbene il Re-ranking a due stadi descritto nel paragrafo precedente ottimizzi l'ordinamento lineare dei chunk estratti, esso risente di un limite intrinseco dei database vettoriali e lessicali tradizionali: l'incapacità di tracciare relazioni ipertestuali esplicite e collegamenti logici trasversali che collegano sezioni distanti della documentazione.

All'interno di un manuale di manutenzione industriale come quello del controller **Fanuc R-30iB Mate**, le informazioni critiche per la risoluzione di un guasto sono intrinsecamente connesse: un codice d'allarme stampato nella sezione di diagnostica rimanda a una scheda elettronica descritta nel capitolo dell'hardware, la quale a sua volta fa riferimento a un connettore specifico o a una procedura di sostituzione. Per catturare questa rete di interdipendenze, il sistema ARIS affianca alla pipeline ibrida un **Grafo di Conoscenza relazionale** (*Knowledge Graph*) costruito in modalità *lightweight*³ direttamente sui metadati estratti dai chunk.

³leggero in termini di memoria, di moderata complessità

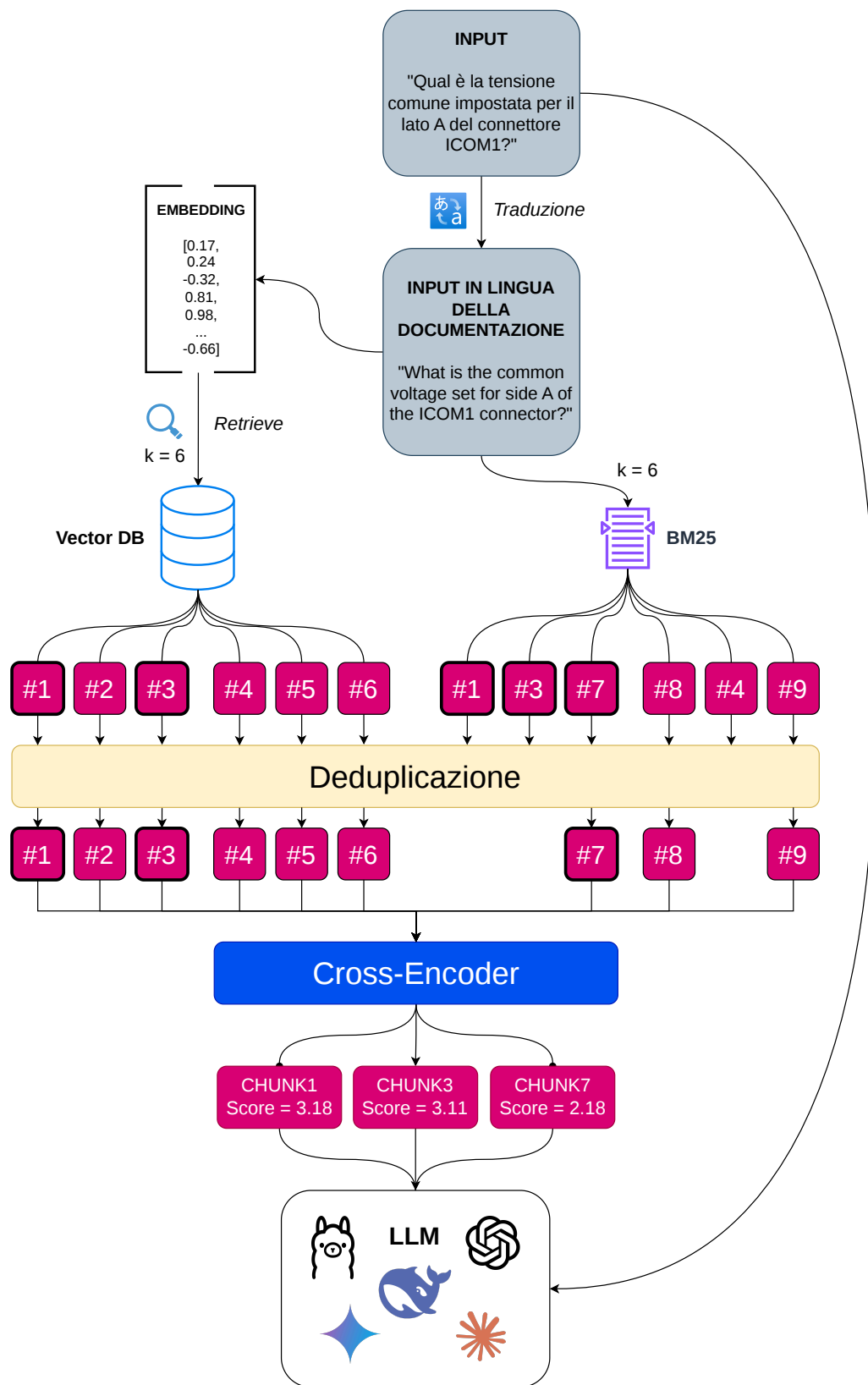


Figura 3.5: Schema pipeline Rerank RAG

3.2.4.1 Topologia del Grafo Relazionale e Rilevamento delle Entità

Il modello poggia sulla libreria `NetworkX`⁴ per la strutturazione di un grafo non orientato $G = (V, E)$, in cui l'insieme dei nodi V rappresenta la totalità dei chunk documentali indicizzati. La generazione degli archi E (i collegamenti) e il calcolo dei relativi pesi matematici riflettono due distinte logiche ingegneristiche:

1. **Connessione per Adiacenza Sequenziale (Struttura del Manuale):** Il sistema raggruppa i nodi appartenenti al medesimo file e ne ordina le pagine. Se due chunk appartengono a pagine adiacenti ($\Delta_{\text{page}} \leq 1$), viene iniettato un arco con peso $w = 0.5$ contrassegnato dal metadato "adjacency". Questa logica preserva la continuità narrativa delle procedure descritte su più facciate consecutive.
2. **Connessione per Entità Condivise (Logica Diagnostica):** Durante la fase di ingestion, il testo di ciascun chunk viene scansionato mediante espressioni regolari (*Regex Mapping*) per estrarre tre specifiche classi di entità industriali:
 - *Codici d'allarme:* Espressioni del tipo `[A-Z]{4}-\d{3}` (es. SRV0-001, SRV0-408).
 - *Codici componente (Part Numbers):* Stringhe alfanumeriche complesse identificative dei ricambi Fanuc (es. A05B-2600-J570, A20B-8200-0790).
 - *Identificativi di connettori:* Sigle delle interfacce di cablaggio (es. CRMA15, CRMA16, JD1A).

Qualora due o più chunk posizionati in punti diversi del database condividano una o più entità tecniche, il sistema traccia un arco diretto tra i nodi, incrementando il peso dell'arco di 1.0 per ogni entità in comune e immagazzinando i token identificativi all'interno dell'attributo `entities`.

Questo meccanismo di interconnessione genera una topologia di rete estremamente significativa, i cui riscontri applicativi nell'interfaccia di ARIS sono visibili nelle figure seguenti.

L'analisi visiva della Figura ?? evidenzia l'emergere spontaneo di strutture a cluster: i chunk appartenenti allo stesso manuale (identificati dal medesimo codice colore, come il cluster centrale verde associato a `connections.pdf`) tendono a raggrupparsi densamente.

Al contempo, l'ispezione ravvicinata della Figura ?? mostra la potenza diagnostica dell'architettura: partendo da un chunk focalizzato sulla gestione dell'arresto controllato (`safety_precautions.pdf`, pagina s-11), il grafo irradia una serie di archi a stella verso nodi esterni che trattano i codici d'errore della scheda d'emergenza (SRV0-001, SRV0-002, SRV0-007). Questa struttura consente al sistema di superare la distanza puramente lessicale o semantica delle query utente.

3.2.4.2 Algoritmo di Espansione e Funzionamento del Codice

Il modulo `GraphExpandedRetriever` governa il recupero dell'informazione a runtime eseguendo una pipeline a quattro stadi rigidamente sequenziali:

1. **Estrazione del Pool Base Allargato:** La query dell'utente viene inviata al retriever ibrido impostando un fattore di recupero pari a $k_{\text{base}} = 12$. Lo script colleziona i candidati e ne rimuove i duplicati testuali.

⁴libreria Python usata per creare, manipolare e studiare le strutture complesse e dinamiche come i grafi

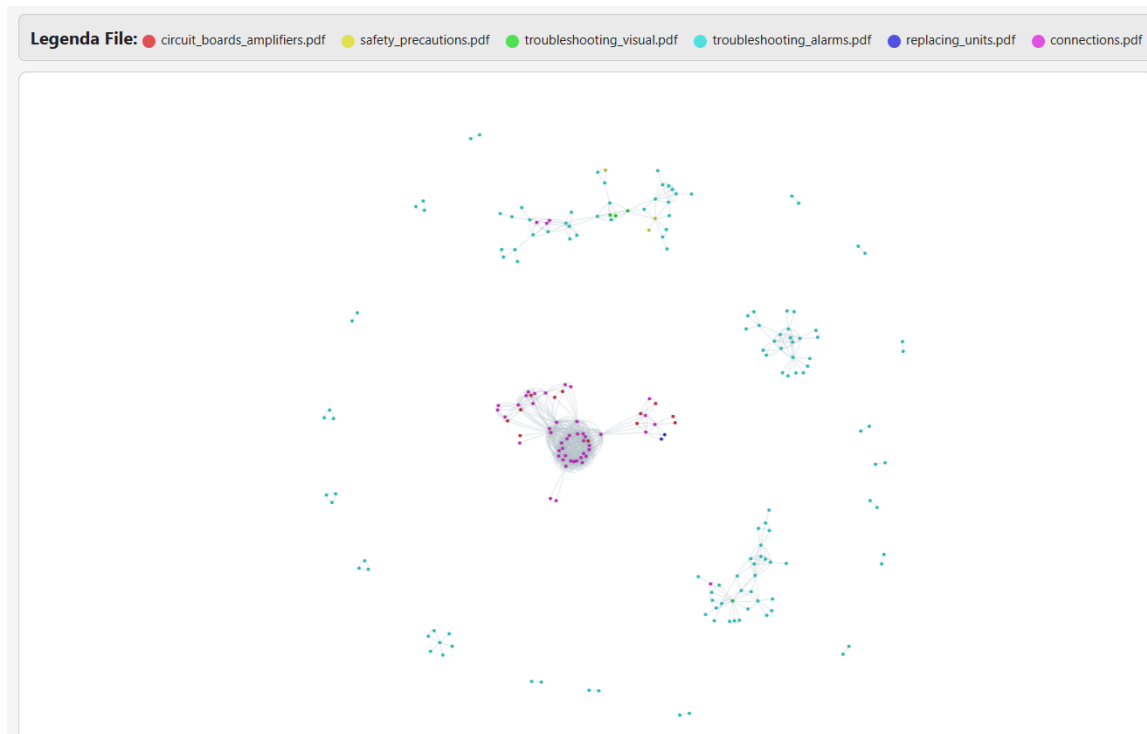


Figura 3.6: Mappa Relazionale globale del GraphRAG di ARIS. Si osservi la presenza di macro-cluster isolati che rispecchiano le singole sezioni della manualistica.

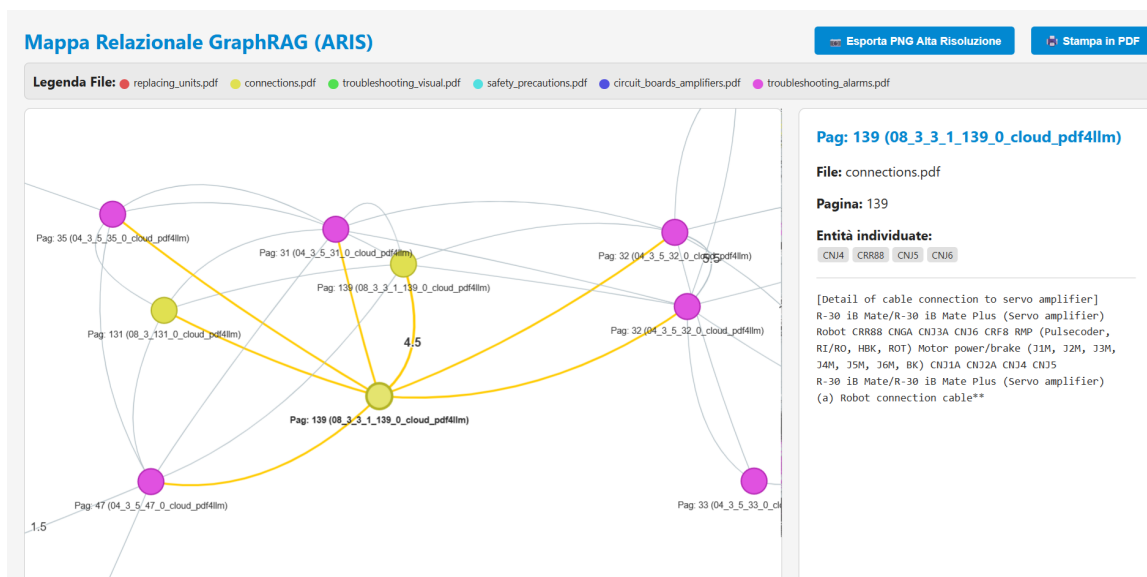


Figura 3.7: Dettaglio microscopico di un'espansione relazionale nel grafo per una sezione di connessioni (*connections.pdf*).

2. **Primo Reranking di Selezione:** Il pool de-duplicato viene valutato dal Cross-Encoder. Il sistema seleziona i primi `top_n` chunk (impostato a quota 4), identificando i quattro nuclei informativi a massima rilevanza diretta rispetto alla domanda.
3. **Espansione Relazionale sul Grafo:** Per ciascuno dei 4 chunk promossi, lo script interroga la struttura G di NetworkX, estraendo l'elenco dei nodi vicini (*neighbors*). Tra tutti i vicini disponibili, l'algoritmo seleziona e aggrega al pool solo il nodo che esibisce il peso dell'arco (*weight*) più elevato, ovvero il frammento documentale che condivide il maggior numero di entità o relazioni con il chunk di partenza.
4. **Reranking Finale Combinato:** I chunk base selezionati e i rispettivi migliori vicini recuperati dal grafo vengono uniti in un unico pool espanso (avente una dimensione massima di 8 chunk unici). Questo set finale viene sottoposto a una seconda e definitiva valutazione da parte del Cross-Encoder, che riordina l'intera graduatoria restituendo all'LLM i migliori $k_{final} = 4$ frammenti.

La traduzione software di questa complessa logica di recupero ed espansione ipertestuale è riportata nel Listing 3.5.

Listing 3.5: Implementazione delle classi `LightweightGraphRAG` e `GraphExpandedRetriever` per la gestione e l'espansione del Grafo della Conoscenza.

```

1 class GraphExpandedRetriever:
2     """Wrapper Retriever LangChain compatibile che implementa l'
3     espansione dei chunk via Grafo."""
4     def __init__(self, base_retriever, graph_rag, k_base: int = 12,
5     top_n: int = 4, debug: bool = False):
6         self.base_retriever = base_retriever
7         self.graph_rag = graph_rag
8         self.k_base = k_base
9         self.top_n = top_n
10        self.debug = debug
11
12    def invoke(self, query: str) -> list:
13        # 1. Recupera un pool più ampio dall'ibrido base (k_base =
14        12)
15        base_docs = self.base_retriever.invoke(query)[:self.k_base]
16        if not base_docs:
17            return []
18
19        # Rimozione duplicati basati sul contenuto testuale
20        seen = set()
21        unique_base_docs = []
22        for doc in base_docs:
23            if doc.page_content not in seen:
24                seen.add(doc.page_content)
25                unique_base_docs.append(doc)
26
27        # 2. Primo Rerank sui chunk base per selezionare i top_n base
28        docs (top_n = 4)

```

```

25     pairs = [[query, doc.page_content] for doc in
26         unique_base_docs]
27     scores_base = self.graph_rag.reranker.predict(pairs)
28     scored_base_docs = sorted(zip(scores_base, unique_base_docs),
29         key=lambda x: x[0], reverse=True)
30     selected_base = [doc for score, doc in scored_base_docs[:self
31         .top_n]]
32
33     # 3. Espansione trasversale tramite il Grafo per tutti i
34     selected_base
35     candidate_pool = list(selected_base)
36     final_keys = {_get_node_key(doc) for doc in selected_base}
37
38     for doc in selected_base:
39         rk = _get_node_key(doc)
40         if not self.graph_rag.G.has_node(rk):
41             continue
42
43         # Identificazione dei nodi vicini non ancora inclusi nel
44         pool
45         neighbors_with_weights = []
46         for neighbor in self.graph_rag.G.neighbors(rk):
47             if neighbor not in final_keys:
48                 edge_data = self.graph_rag.G.get_edge_data(rk,
49                     neighbor)
50                 weight = edge_data.get("weight", 1.0)
51                 neighbors_with_weights.append((neighbor, weight))
52
53         # Selezione deterministica del miglior vicino (arco a
54         massimo peso)
55         best_neighbor = None
56         if neighbors_with_weights:
57             best_neighbor = sorted(neighbors_with_weights, key=
58                 lambda x: x[1], reverse=True)[0][0]
59
60         if best_neighbor:
61             final_keys.add(best_neighbor)
62             candidate_pool.append(self.graph_rag.chunk_dict[
63                 best_neighbor])
64
65     # 4. Secondo Rerank sul pool combinato (base + vicini del
66     grafo)
67     seen_pool = set()
68     unique_candidate_pool = []
69     for doc in candidate_pool:
70         if doc.page_content not in seen_pool:
71             seen_pool.add(doc.page_content)
72             unique_candidate_pool.append(doc)
73
74     pairs_pool = [[query, doc.page_content] for doc in

```

```

        unique_candidate_pool]
65     scores_pool = self.graph_rag.reranker.predict(pairs_pool)
66     scored_pool = sorted(zip(scores_pool, unique_candidate_pool),
        key=lambda x: x[0], reverse=True)
67
68     # Restituisce all'LLM l'insieme a massima pertinenza (k_final
        = 4)
69     return [doc for score, doc in scored_pool[:self.top_n]]

```

Grazie a questo impianto a doppio stadio neurale intervallato da un'estrazione topologica, il sistema ARIS si dimostra in grado di "iniettare" nel prompt informazioni che un sistema RAG tradizionale avrebbe inevitabilmente escluso. Se un manutentore interroga l'assistente su un componente specifico, il sistema non si limita a estrarre la scheda descrittiva, ma recupera autonomamente dal grafo le pagine relative ai connettori fisici associati a quella scheda, azzerando i tassi di omissione informativa (*Context Recall*) e blindando la successiva fase di inferenza del modello.

3.3 Prompt Engineering e Orchestrazione dei Modelli

A valle delle procedure di estrazione avanzata dei contesti, la determinazione dell'accuratezza, della sicurezza e della forma dell'output finale restituito all'operatore sul campo dipende interamente dalle fasi di *Prompt Engineering* e dall'orchestrazione sincrona dei Large Language Models. Questa sezione documenta l'ingegnerizzazione dei prompt d'istruzione e di classificazione, le configurazioni di instradamento dei server di inferenza sia in ambiente Cloud sia On-Premise, e l'infrastruttura di telemetria utilizzata per il monitoraggio a runtime delle chiamate.

3.3.1 Ingegnierizzazione dei Prompt e Regole di Grounding

Il comportamento cognitivo del sistema ARIS è governato da due tipologie di prompt ingegnerizzati: un prompt di smistamento a monte (*Classification Prompt*), deputato all'identificazione dell'intento dell'operatore, e un prompt di generazione a valle (*System Prompt*), specializzato nel vincolare la risposta finale all'autenticità dei manuali tecnici (*Grounding*).

3.3.1.1 Il Prompt di Classificazione degli Intenti (Query Routing)

Prima di attivare i moduli di recupero lessicale o vettoriale, il sistema deve comprendere la natura della richiesta per evitare uno spreco di risorse computazionali. A questo scopo, l'algoritmo interroga il modulo `llm_classifier` attraverso un prompt di classificazione rigido e binario.

L'obiettivo di questo prompt è classificare la stringa di input in due sole categorie mutualmente esclusive: **TECHNICAL** o **GENERAL**. Le istruzioni operative fornite al modello all'interno del codice sono strutturate come segue:

- **TECHNICAL:** Identifica qualsiasi quesito inerente alla diagnostica dei guasti, interpretazione dei codici d'allarme (es. `SRV0-004`), schemi di cablaggio, specifiche hardware dei moduli o procedure di manutenzione dei robot Fanuc. L'assegnazione di questo

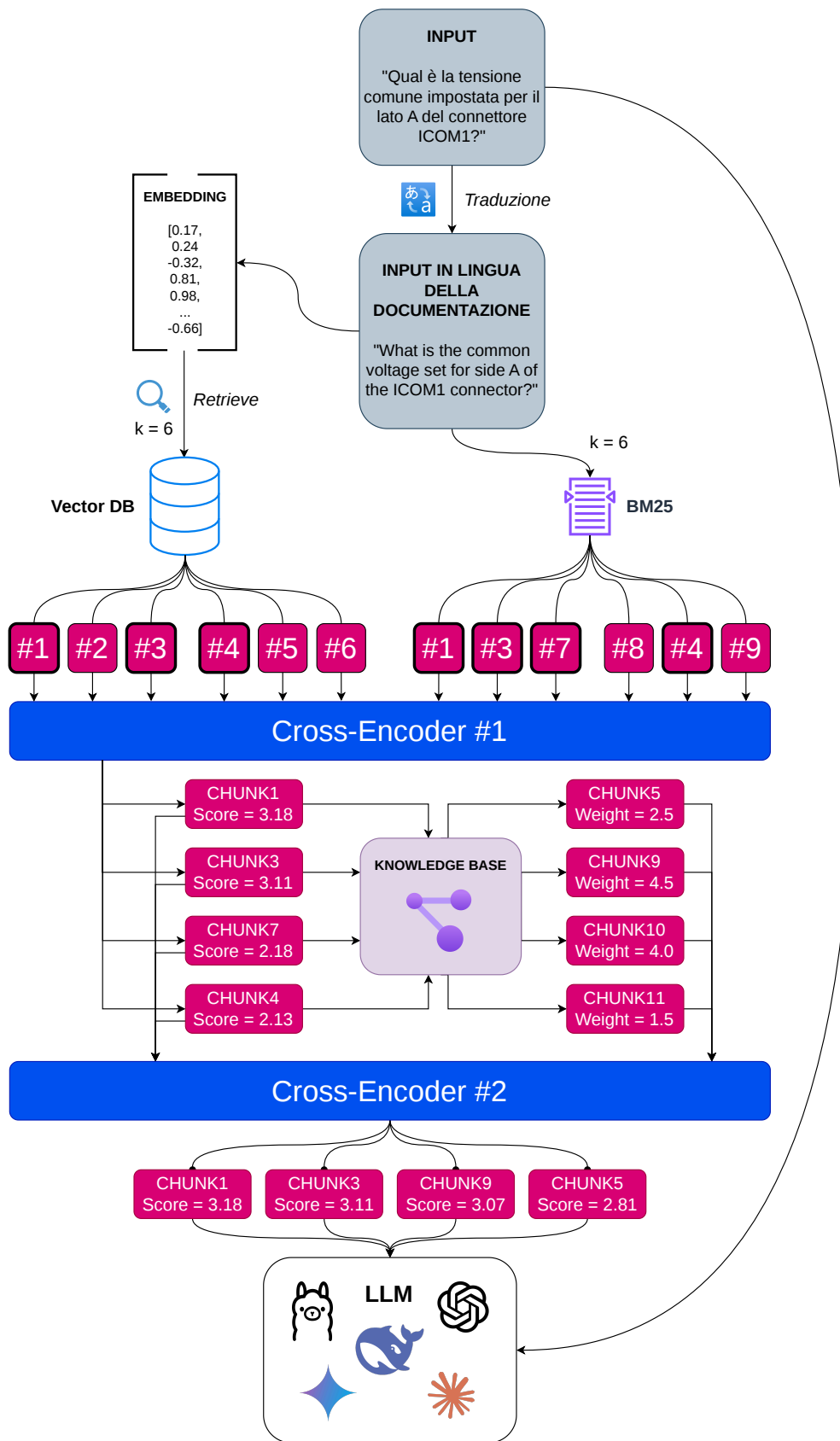


Figura 3.8: Schema pipeline Graph RAG

tag indica al backend di attivare l'intera pipeline di recupero (*Ensemble Retriever* ed eventuale *Re-ranking* o *GraphRAG*).

- **GENERAL:** Intercetta i messaggi di cortesia, i saluti dell'operatore (es. "ciao", "buongiorno"), le presentazioni, le domande sulle capacità dell'assistente (es. "chi sei?") o qualsiasi argomento estraneo al perimetro robotico. L'assegnazione di questo tag comanda alla pipeline di bypassare completamente la fase di retrieval, instradando l'input direttamente verso una catena di generazione standard (`chain_general`) per rispondere in modo naturale e cordiale.

Per azzerare l'introduzione di metatesto o commenti da parte del classificatore, il prompt impone una regola sintattica stringente: *"Rispondi ESCLUSIVAMENTE con una delle due parole: 'TECHNICAL' o 'GENERAL'".* L'adozione di un output a singolo token deterministico rende l'operazione di query routing estremamente veloce e robusta in fase di parsing del codice.

3.3.1.2 Il System Prompt e le Regole di Ancoraggio Fattuale

Una volta determinato l'intento ed effettuato il recupero (esclusivamente per le richieste classificate come **TECHNICAL**), entra in gioco il *System Prompt* principale. Per garantire l'affidabilità delle risposte all'interno di un ecosistema industriale — dove un'istruzione errata può compromettere l'integrità dei macchinari — il prompt è progettato per imporre un vincolo assoluto di ancoraggio fattuale ai soli chunk estratti.

Il prompt organizza la logica di generazione secondo tre regole di comportamento:

1. **Gestione della Chat Generica:** Regola l'interazione standard del robot per i messaggi classificati come **GENERAL**. Il modello agisce come un assistente amichevole e disponibile per i robot Fanuc, formulando risposte cordiali e descrivendo brevemente le proprie funzioni di supporto alla manutenzione.
2. **Gestione della Cronologia Conversazionale:** Istruisce il modello a ispezionare dinamicamente la memoria degli ultimi scambi memorizzati nel prompt (`{history}`), consentendo all'operatore di formulare domande di follow-up o richieste di approfondimento contestualizzate (ad esempio, "puoi approfondire l'ultimo punto?" o "come dicevi prima...").
3. **Ancoraggio alle Fonti Tecniche:** Per le richieste di manutenzione, il modello è obbligato a estrarre le risposte tassativamente dal perimetro del `{context}`. Poiché la documentazione Fanuc contiene ampie tabelle diagnostiche, il prompt impone di analizzare il contesto riga per riga per individuare le corrispondenze esatte. Solo se il dato è assente nel testo recuperato, il modello deve declinare la risposta tramite la stringa tassativa: "La documentazione disponibile non contiene informazioni sufficienti. Si consiglia di consultare un tecnico qualificato".

Oltre alle regole di grounding, il System Prompt impone precisi vincoli di formattazione: le risposte agli allarmi devono seguire una struttura numerata rigorosa (1. Significato, 2. Possibili cause, 3. Controlli, 4. Azioni, 5. Fonte), mentre le risposte sulle specifiche dei componenti adottano uno stile discorsivo. Viene infine configurato un obbligo stringente

di citazione dei metadati delle fonti in calce all'output, forzando l'LLM a compilare la riga finale nel formato standardizzato: "Fonte documentale: [nome_file.pdf] (Pagina: [numero_pagina])".

3.3.2 Integrazione alle API Cloud (OpenRouter) e On-Premise (LM Studio)

L'orchestrazione dei flussi di inferenza adotta l'architettura a tre istanze client differenziate analizzata nel Capitolo 2. L'ecosistema ARIS istanzia infatti tre oggetti indipendenti della classe `ChatOpenAI` di `LangChain`, ottimizzati ciascuno per compiti specifici attraverso la modulazione degli iperparametri di generazione:

1. **llm_classifier:** Dedicato esclusivamente alla categorizzazione semantica immediata delle query e ai layer di traduzione. È configurato a temperatura $\tau = 0.0$ per massimizzare il determinismo strutturale e con una finestra di output ridotta a un massimo di 1024 token per minimizzare i tempi di calcolo.
2. **llm_technical:** Il motore deputato alla formulazione delle procedure di manutenzione complesse. Opera a temperatura $\tau = 0.0$ per blindare il grounding ed evitare variazioni creative delle risposte tecniche, attivando al contempo lo *streaming* dei token per migliorare la fluidità dell'interfaccia utente.
3. **llm_general:** Utilizzato per le risposte di cortesia della chat generica. In questo caso la temperatura è elevata a $\tau = 0.7$, iniettando la flessibilità linguistica necessaria a una conversazione naturale.

La flessibilità infrastrutturale del sistema consente di variare runtime l'ambiente di deployment — Cloud o locale On-Premise — modificando unicamente i parametri di base-URL e chiave API all'interno dello script di avvio, come illustrato nel Listing 3.6.

Listing 3.6: Configurazione e orchestrazione dinamica dei client LLM per ambienti di calcolo Cloud e Locali.

```

1 def setup_rag_chain(retriever, env="locale", model_name=None):
2     if env == "locale":
3         model = model_name if model_name else os.getenv("
4             LOCAL_LLM_MODEL", None)
5         base_url = os.getenv("LOCAL_LLM_URL", "http://localhost:1234/
6             v1")
7         # Client locale interfacciato con l'endpoint locale di LM
8         Studio
9         llm_technical = ChatOpenAI(
10             model=model,
11             base_url=base_url,
12             api_key="lm-studio",
13             temperature=0.0,
14             streaming=True,
15             max_tokens=2048
16         )

```

```

14         # [... inizializzazione llm_classifier ed llm_general su
           localhost ...]
15     elif env == "cloud":
16         model = model_name if model_name else "openai/gpt-4o-mini"
17         base_url = "https://openrouter.ai/api/v1"
18         api_key = os.getenv("OPENAI_API_KEY")
19         # Client cloud instradato verso l'aggregatore di API
           OpenRouter
20         llm_technical = ChatOpenAI(
21             model=model,
22             openai_api_base=base_url,
23             openai_api_key=api_key,
24             temperature=0.0,
25             streaming=True
26         )
27         # [... inizializzazione llm_classifier ed llm_general su
           cloud ...]

```

Come si evince dall'estrazione del listato, quando il sistema opera in modalità "locale", l'orchestrazione converge verso il server locale esposto su `localhost:1234` gestito dall'applicazione **LM Studio**, la quale alloca i pesi del modello `phi3.5` ottimizzando l'uso dei thread CPU. In modalità "cloud", l'endpoint viene re-indirizzato verso il gateway di **OpenRouter**, sfruttando le API di OpenAI per l'interrogazione del modello di frontiera `gpt-4o-mini`.

3.3.3 Strumentazione di Telemetria e Monitoraggio con LangSmith

Per tracciare lo stato di esecuzione delle catene, validare la correttezza del prompt engineering ed eseguire il debug visivo dei token processati, l'intero backend di ARIS è stato agganciato alla piattaforma di telemetria **LangSmith**. Questo strumento intercetta in tempo reale l'albero delle invocazioni (*Trace Graph*) generato dall'Expression Language di LangChain, registrando l'esatto payload di input, la stringa formattata risultante dalla compilazione dei template e i singoli chunk promossi dal retriever.

Un esempio concreto di ispezione di un tracciato di telemetria a runtime è documentato dal blocco logico strutturato estratto dalla console e riportato nel Listing 3.7.

Listing 3.7: Payload di telemetria estratto da LangSmith per una query tecnica di estrazione moduli.

```

{
  "run_inputs": {
    "query": "What are the steps to follow to extract a module from
             the Fanuc R-30iB controller?"
  },
  "retrieved_context": [
    {
      "metadata": {
        "chunk_id": "doc_connections_page186_0_cloud_pdf411m",
        "file_name": "connections.pdf",
        "page": "186",
        "token_size_target": 700
      }
    }
  ]
}

```

```

    },
    "page_content": "[EXTERNAL VIEW OF THE CONTROLLER]\nEmergency
    stop button Battery mode switch Breaker Main board Back
    plane unit Heat exchanger 6-Axis Servo amplifier E-stop unit
    \n[Caption]: Fig.2.1 \n(c) R-30 iB Mate cabinet interior (
    Front-1) Noise Filter (EMC Option) Power supply unit Process
    I/O board (Option)..."
  }
]
}

```

L'analisi del payload nel Listing 3.7 evidenzia l'efficacia d'insieme del monitoraggio: a fronte di una query complessa focalizzata sulla scomposizione hardware del cabinet Fanuc ("What are the steps to follow to extract a module..."), LangSmith intercetta l'avvenuta esecuzione della catena evidenziando l'avvenuto grounding.

Il sistema mostra che il retriever ha estratto con successo il chunk identificato univocamente dall'ID `doc_connections_page186_0`, specificando nei metadati l'origine esatta dal file `connections.pdf` alla pagina 186. L'ispezione del `page_content` conferma la corretta inclusione della mappatura dei componenti fisici della scheda madre (*Main board*), dell'unità posteriore (*Back plane unit*) e dell'amplificatore del servo-asse (*6-Axis Servo amplifier*).

Questa visibilità granulare sui dati in transito ha permesso di tarare empiricamente le soglie del re-ranker, verificare che le tabelle Markdown non subissero troncamenti all'interno del prompt e monitorare i consumi esatti dei token per singola chiamata, ponendo le basi quantitative per le estese attività di validazione e calcolo delle metriche RAGAS che verranno approfondite all'interno del Capitolo 4.

3.4 Interfaccia Utente e Gestione della Conversazione

L'ultimo livello dell'architettura del sistema ARIS poggia sullo sviluppo dell'interfaccia utente finale (*Frontend*). In un contesto industriale, l'accessibilità e la rapidità di interazione con un assistente virtuale sono requisiti stringenti: un manutentore d'impianto necessita di un'applicazione snella, reattiva e in grado di operare direttamente sui terminali di linea o dispositivi mobili senza richiedere complesse installazioni software locali. I prossimi paragrafi documentano l'ingegnerizzazione dell'interfaccia grafica e le soluzioni tecniche adottate per la gestione della persistenza dei dati a runtime.

3.4.1 Architettura e Presentazione dell'Applicazione Web

Per la realizzazione dell'interfaccia grafica è stato selezionato il framework open-source **Streamlit**, una libreria d'elezione per il *rapid prototyping* di applicazioni web orientate alla data science. La struttura dell'interfaccia utente progettata per ARIS è visibile nello screenshot della console riportato nella Figura 3.9.

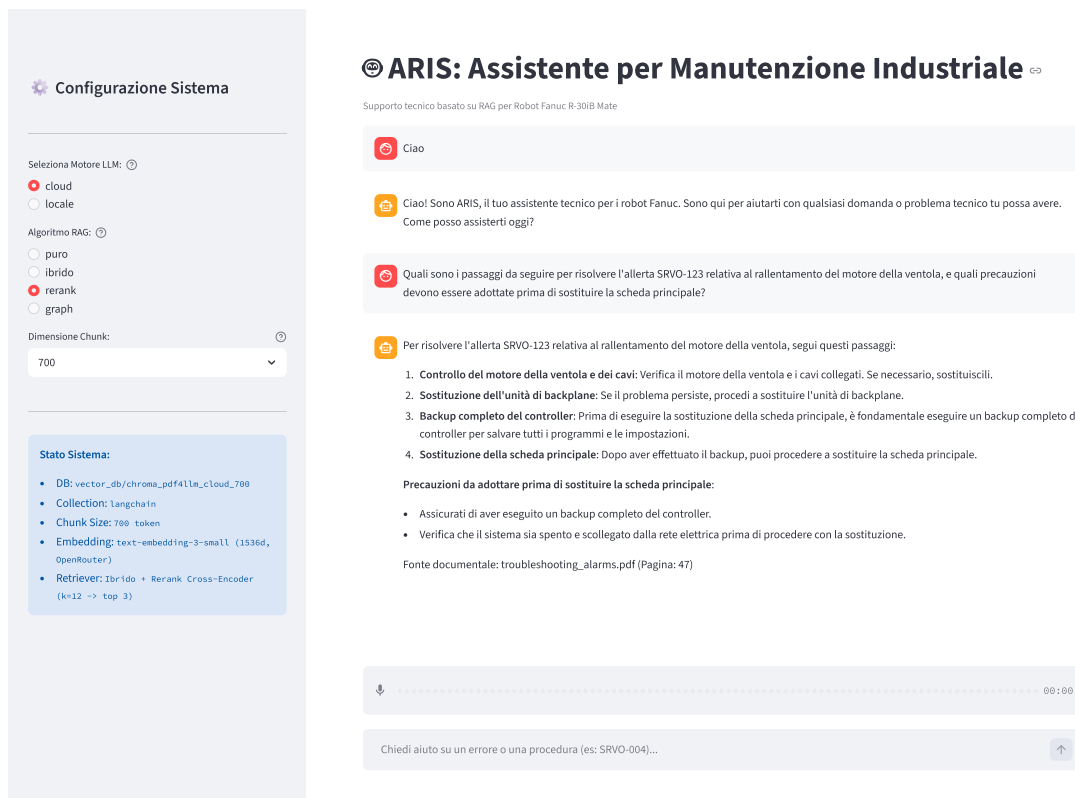


Figura 3.9: *Interfaccia grafica dell'applicazione ARIS sviluppata in ambiente Streamlit. Sulla barra laterale sinistra sono visibili i moduli di configurazione iperparametrica, mentre a destra si sviluppa il pannello interattivo di messaggistica.*

L'applicazione poggia su un layout a due aree funzionali distinte:

- **Pannello di Configurazione (Sidebar):** Posizionato sul lato sinistro, consente all'operatore di impostare runtime le variabili core della pipeline, selezionando il motore LLM (Cloud via OpenRouter o Locale tramite LM Studio), l'algoritmo di recupero (*puro*, *ibrido*, *rerank* o *graph*) e la dimensione geometrica dei chunk (300, 700 o 1000 token). Al di sotto dei selettori, un riquadro informativo (`st.sidebar.info`) espone in tempo reale lo stato interno del sistema, tracciando il percorso esatto del Vector DB caricato in memoria e il modello di embedding attivo.
- **Pannello di Interazione (Chat Interface):** Sviluppato sull'area principale a destra, emula un'interfaccia di messaggistica istantanea. Ogni interazione è scandita da icone differenziate per l'operatore (colore rosso) e per l'assistente (colore arancione). La risposta tecnica generata, visibile nell'esempio per la risoluzione dell'allarme SRVO-123, rispetta i vincoli di formattazione strutturata imposti dal prompt engineering, restituendo un elenco numerato delle azioni di manutenzione ed esplicitando la fonte bibliografica di ancoraggio (`troubleshooting_alarms.pdf` (Pagina: 47)).

3.4.2 Gestione della Conversational History e Riconoscimento Vocale

L'ingegnerizzazione del codice sorgente di ARIS affronta due problemi critici legati alla stabilità del frontend: l'assenza intrinseca di memoria del protocollo HTTP (che causerebbe la

perdita della chat ad ogni aggiornamento della pagina) e l'ottimizzazione dell'interazione sul campo in ambienti industriali rumorosi.

3.4.2.1 Persistenza della Memoria e Caching delle Risorse

Per preservare la cronologia della conversazione durante il ciclo di esecuzione dell'applicazione, il software si appoggia all'oggetto globale `st.session_state`. All'interno dello stato di sessione viene inizializzato l'array `messages`, il quale memorizza in un dizionario il ruolo (*user* o *assistant*) e il contenuto testuale di ogni messaggio.

Prima dell'inoltro della domanda alla pipeline RAG, il sistema estrae dinamicamente dal `session_state` gli ultimi 3 scambi conversazionali (corrispondenti a un massimo di 6 messaggi totali), escludendo la query corrente. Questi record vengono concatenati in un'unica stringa formattata (`history_str`) e iniettati nello spazio `{history}` del prompt, consentendo all'LLM di mantenere il filo logico del discorso senza saturare la finestra di contesto con messaggi obsoleti.

Un secondo accorgimento ingegneristico poggia sull'uso del decoratore `@st.cache_resource` applicato alla funzione di attivazione della catena RAG. Questa direttiva istruisce Streamlit a mantenere persistenti in memoria l'istanza di ChromaDB e il modello neurale di Re-ranking, evitando l'esecuzione del costoso setup software ad ogni invio di messaggi da parte dell'utente.

3.4.2.2 Integrazione del modulo Speech-to-Text

Per agevolare l'operatore sul campo, spesso impossibilitato all'uso estensivo della tastiera durante le manovre di riparazione del robot, l'applicazione integra un layer di input vocale (`st.audio_input`) basato sulla libreria `SpeechRecognition`. Il sistema cattura la registrazione audio, instradandola verso i motori di decodifica acustica per estrarre la stringa testuale in lingua italiana mediante API `Speech-to-Text`⁵. Il testo trascritto viene automaticamente iniettato nella variabile `prompt`, sovrascrivendo l'input da tastiera.

La porzione saliente del codice Python deputata all'orchestrazione del frontend, alla gestione del caching e all'alimentazione in streaming della chat è riportata nel Listing 3.8.

Listing 3.8: Codice sorgente Python per l'inizializzazione iperparametrica e la gestione dei messaggi in streaming nell'interfaccia Streamlit di ARIS.

```

1 @st.cache_resource
2 def init_rag_chain(env: str, chunk_size: int, rag_type: str):
3     """Inizializza l'intera chain RAG caricando il Vector DB e i
4         relativi moduli."""
5     db_path = os.path.join(PROJECT_ROOT, get_db_path(env, chunk_size))
6     embedder = get_embeddings(env)
7     lc_chroma = Chroma(
8         persist_directory=db_path,
9         collection_name=COLLECTION_NAME,
10        embedding_function=embedder
11    )
12    # Selezione dinamica del Retriever in base all'algoritmo
13    configurato
14    if rag_type == "puro":

```

⁵Tecnica di Artificial Intelligence che converte audio in testo (trascrizione)

```

13     retriever = lc_chroma.as_retriever(search_kwargs={"k": 3})
14     return rag_pipeline_hybrid.setup_rag_chain(retriever, env=env)
15 elif rag_type == "rerank":
16     ensemble = rag_pipeline_rerank.build_hybrid_retriever(
17         lc_chroma, k=6)
18     retriever = rag_pipeline_rerank.HybridRerankRetriever(
19         ensemble, top_n=3)
20     return rag_pipeline_rerank.setup_rag_chain(retriever, env=env)
21 # [...] gestione degli altri algoritmi 'ibrido' e 'graph' [...]
22 # --- GESTIONE DELLA CRONOLOGIA E GENERAZIONE STREAMING ---
23 if prompt:
24     st.session_state.messages.append({"role": "user", "content":
25         prompt})
26     with st.chat_message("user"):
27         st.markdown(prompt)
28
29     with st.chat_message("assistant"):
30         with st.spinner("L'IA sta consultando i manuali..."):
31             # Estrazione e formattazione della history degli ultimi 3
32             # scambi
33             history_msgs = st.session_state.messages[: -1][ -6:]
34             history_str = "\n".join(f"{m['role'].upper()}: {m['content']}" for m in history_msgs) if history_msgs
35             else "(nessuna conversazione precedente)"
36
37             # Attivazione dello streaming dei token per una UX
38             # reattiva
39             response_stream = rag_chain.stream({"question": prompt, "
40                 history": history_str})
41
42             # Estrazione del primo chunk per sbloccare l'interfaccia
43             try:
44                 first_chunk = next(response_stream)
45             except StopIteration:
46                 first_chunk = ""
47
48             def stream_generator():
49                 if first_chunk: yield first_chunk
50                 for chunk in response_stream: yield chunk
51
52             # Scrittura dinamica dei token a schermo
53             full_response = st.write_stream(stream_generator())
54             st.session_state.messages.append({"role": "assistant", "
55                 content": full_response})

```

Come documentato dal listato, l'interfaccia sfrutta il metodo `st.write_stream()` accoppiato a una funzione generatrice (`stream_generator()`). Questa architettura consente di mostrare all'operatore la risposta in tempo reale, carattere per carattere, riducendo a zero la latenza percepita dall'utente. Nel momento in cui il flusso di streaming si conclude, la rispo-

sta completa viene salvata in modo permanente nella cronologia di sessione, completando il ciclo iterativo della pipeline.

Capitolo 4

Sperimentazione, Valutazione e Analisi dei Risultati

4.1 Definizione e Validazione del Dataset di Test

La valutazione quantitativa di un'architettura RAG necessita della definizione di un terreno di verifica sperimentale (*Ground Truth*) composto da un insieme accoppiato di quesiti e risposte attese iper-specifiche. La redazione manuale di un simile benchmark da parte di ingegneri o tecnici manutentori, sebbene accurata, risulta un'operazione estremamente onerosa in termini di tempo e incline a sviste sistematiche o disomogeneità formali.

Al fine di garantire l'imparzialità e la ripetibilità dei test sul sistema ARIS, l'intero processo di generazione, verifica qualitativa, correzione e traduzione del dataset è stato automatizzato mediante lo sviluppo di una pipeline software modulare. Per evitare bias di autovalidazione, il framework applica il principio dell'indipendenza computazionale, separando nettamente i modelli linguistici utilizzati per la creazione della conoscenza da quelli deputati al controllo di qualità e all'inferenza finale.

4.1.1 Architettura della Pipeline di Generazione

Il processo di strutturazione del dataset si articola su quattro script principali agenti in modo sequenziale sulla base documentale pre-elaborata dal parser **pdf4llm**: `genera_domande.py`, `verifica_domande.py`, `sostituisci_domande.py` e `traduci_domande.py`.

L'architettura complessiva poggia su una scomposizione a tre stadi funzionali:

1. **Generazione Distribuitiva Uniforme:** Lo script `genera_domande.py` analizza i chunk strutturati e calcola un passo di campionamento logico per selezionare in modo speculare e uniforme i blocchi di testo lungo l'intera estensione dei manuali Fanuc. Ciascun chunk selezionato viene somministrato al modello di generazione (**gpt-4o-mini**), vincolato a estrarre una domanda e una risposta attesa (*Expected Answer*) deducibili esclusivamente dal testo sorgente. A questo livello, il sistema impone una rigida ripartizione iperparametrica della difficoltà, mappando il dataset sul target programmatico del **30% di domande Low, 50% Medium e 20% Hard**.
2. **Validazione Eterogenea (LLM Judge):** Una volta generato il file di tipo CSV `test_questions_it.csv` composto da 100 quesiti, entra in funzione lo script di verifica. Il controllo di qualità viene delegato a un modello terzo ad alta dimensionalità

parametrica, nello specifico `meta-llama/llama-3.3-70b-instruct` operante a temperatura zero per massimizzare il rigore deterministico. Il *Judge* agisce come un ispettore indipendente: esamina la pagina originaria del manuale e verifica la correttezza grammaticale, l'assenza di allucinazioni estrinseche nella risposta e l'effettiva pertinenza ipertestuale del numero di pagina associato. L'output viene rigettato e marcato come non valido qualora emerga una minima incoerenza fattuale.

3. **Sostituzione In-Place e Traduzione Tecnica:** Lo script `sostituisci_domande.py` risolve in modo dinamico le anomalie intercettate dal Judge. Invece di invalidare l'intero test-set, il software isola il record difettoso, interroga nuovamente la base di conoscenza cercando chunk alternativi pertinenti alla medesima categoria o capitolo, e avvia una rigenerazione mirata *in-place* che conserva intatti l'ID originario, la categoria d'allarme e il livello di difficoltà target. Ottenuta l'approvazione del Judge, il record viene promosso e inoltrato al modulo `traduci_domande.py`, il quale applica un prompt specialistico di traduzione in lingua inglese (`test_questions_en.csv`), vincolato a preservare intatti i codici alfanumerici Fanuc (es. `SRVO`, `OHAL`) e i codici dei componenti fisici.

La differenziazione tassonomica delle difficoltà e delle categorie imposta all'interno dei prompt di generazione riflette le reali necessità operative riscontrate in officina, muovendosi secondo lo schema logico formalizzato nella Tabella 4.1.

Difficoltà	Quota Target	Definizione e Requisiti di Riscontro Documentale
low	30%	Quesiti focalizzati su singoli parametri, definizioni esplicite o valori numerici isolati (es. tensioni nominali).
medium	50%	Richiesta di tracciamento di passaggi sequenziali o procedure standard descritte all'interno dello stesso paragrafo.
hard	20%	Troubleshooting strutturato, schemi di collegamento multi-nodo, o risoluzione di anomalie che impongono vincoli di backup e sicurezza.

Tabella 4.1: *Tassonomia e criteri di partizionamento iperparametrico del Dataset di Test.*

4.1.2 Analisi Campionaria del Ground Truth Consolidato

Il completamento dei cicli di mutua validazione neurale ha prodotto un dataset di 100 domande blindato dal punto di vista dell'ancoraggio documentale. Per apprezzare la diversificazione semantica e la precisione geometrica dei riferimenti ipertestuali generati, la Tabella 4.2 riporta un estratto significativo composto da record reali estratti dal file di verifica `test_questions_it.csv`.

L'ispezione della Tabella 4.2 evidenzia come la pipeline sia stata in grado di mappare con estrema precisione i diversi livelli di complessità concettuale. Domande a bassa difficoltà (come Q004) estraggono valori puntuali ed espliciti dal testo (+24V). Salendo di livello (es. Q033), il dataset introduce l'obbligo di verificare passaggi operativi complessi (la rimozione della guaina del cavo Ethernet, l'esposizione dello schermo e la pressione contro la piastra di terra).

Infine, i quesiti classificati come **hard** (quali Q084 e Q100) si focalizzano sulle reali criticità diagnostiche di officina: essi forzano l'architettura a recuperare non solo l'azione riparativa

diretta (la sostituzione della ventola o della scheda madre), ma anche i vincoli critici di sicurezza correlati, come l'obbligo tassativo di eseguire un backup completo del controller prima di disconnettere i componenti elettronici per preservare i programmi dell'utente. Questo impianto di test costituisce la base analitica per il calcolo delle metriche di precisione e recall che verranno esposte nel prossimo paragrafo.

ID	Categoria	Testo della Domanda di Test	File Manuale Sorgente	Pag.	Diff.
Q004	Cons. tecnica	Qual è la tensione comune impostata per il lato A del connettore ICOM1?	circuit_boards_amplifiers.pdf	100	low
Q005	Procedure	Quale valore deve essere impostato regolando i potenziometri VR1 e VR2 durante l'aggiustamento del guadagno e dell'offset del canale 1?	circuit_boards_amplifiers.pdf	101	low
Q031	Cons. tecnica	Qual è il tipo di cavo raccomandato per migliorare la resistenza al rumore elettrico in un ambiente FA secondo il manuale del controller Fanuc R-30iB?	connections.pdf	198	medium
Q033	Procedure	Quali sono i passaggi da seguire per garantire il corretto bloccaggio e schermatura di un cavo Ethernet secondo il manuale del controller Fanuc R-30iB?	connections.pdf	201	medium
Q038	Procedure	Quali sono le conseguenze dell'abilitazione e disabilitazione del circuito HBK quando è chiuso o aperto, e quali allarmi possono verificarsi in ciascuna situazione?	connections.pdf	214	medium
Q069	Procedure	Quali sono i passaggi da seguire per estrarre un modulo dal controller Fanuc R-30iB?	replacing_units.pdf	115	medium
Q084	Troubleshoot.	Quali sono i passaggi da seguire se si verifica l'allerta SRVO-015 a causa di un surriscaldamento del sistema, e quali precauzioni devono essere adottate prima di sostituire la scheda principale?	troubleshooting_alarms.pdf	30	hard
Q100	Troubleshoot.	Quali sono i passaggi da seguire per risolvere l'allerta SRVO-123 relativa al rallentamento del motore della ventola, e quali precauzioni devono essere adottate prima di sostituire la scheda principale?	troubleshooting_alarms.pdf	47	hard

Tabella 4.2: Estratto campionario delle coppie di validazione componenti il *Ground Truth* finale del sistema ARIS.

4.2 Valutazione Quantitativa del Retrieval

La prima fase di validazione si focalizza sull'accuratezza geometrica e topologica dei database vettoriali e lessicali interpellati prima dell'intervento dell'LLM. I test sono stati eseguiti sul dataset di 100 domande combinando i due metodi di parsing del testo (*Euristico* e *pdf4llm*) sia in ambiente locale sia cloud, al fine di mappare il comportamento delle quattro strategie core (*Vector*, *Hybrid*, *Rerank*, *Graph*) rispetto alle metriche di **Hit Rate@3** e **Mean Reciprocal Rank (MRR)**.

L'ispezione della Figura 4.1 evidenzia l'andamento parabolico e incrementale legato alla complessità algoritmica dei retriever:

- **Efficacia del Two-Stage Retrieval (Rerank):** In tutte le configurazioni d'esame, l'introduzione del modulo Cross-Encoder (*Rerank*) determina il picco prestazionale assoluto, registrando un Hit Rate@3 dell'**83.0%** e un MRR pari a **0.750** nell'istanza basata su *pdf4llm_cloud_700*. Il riordinamento basato sull'attenzione incrociata termine-per-termine permette di sanare i limiti dei modelli Bi-Encoder, portando in cima alla classifica i chunk a massima densità informativa.
- **Il comportamento di GraphRAG (Graph):** Si osserva come la strategia *Graph* sperimenti una lieve contrazione dell'Hit Rate rispetto al *Rerank* puro (scendendo, ad esempio, dall'83.0% all'81.0% su *pdf4llm_cloud_700*). Questo fenomeno è legato alla natura stessa dell'espansione relazionale: l'iniezione nel pool dei nodi vicini (pagine adiacenti o schede correlate) aumenta la copertura informativa complessiva, ma a livello di puro posizionamento geometrico può scalzare un chunk parzialmente rilevante dal podio dei primi tre elementi, alterando la metrica rigida dell'Hit Rate pur mantenendo un MRR quasi identico (**0.752** vs **0.750**).

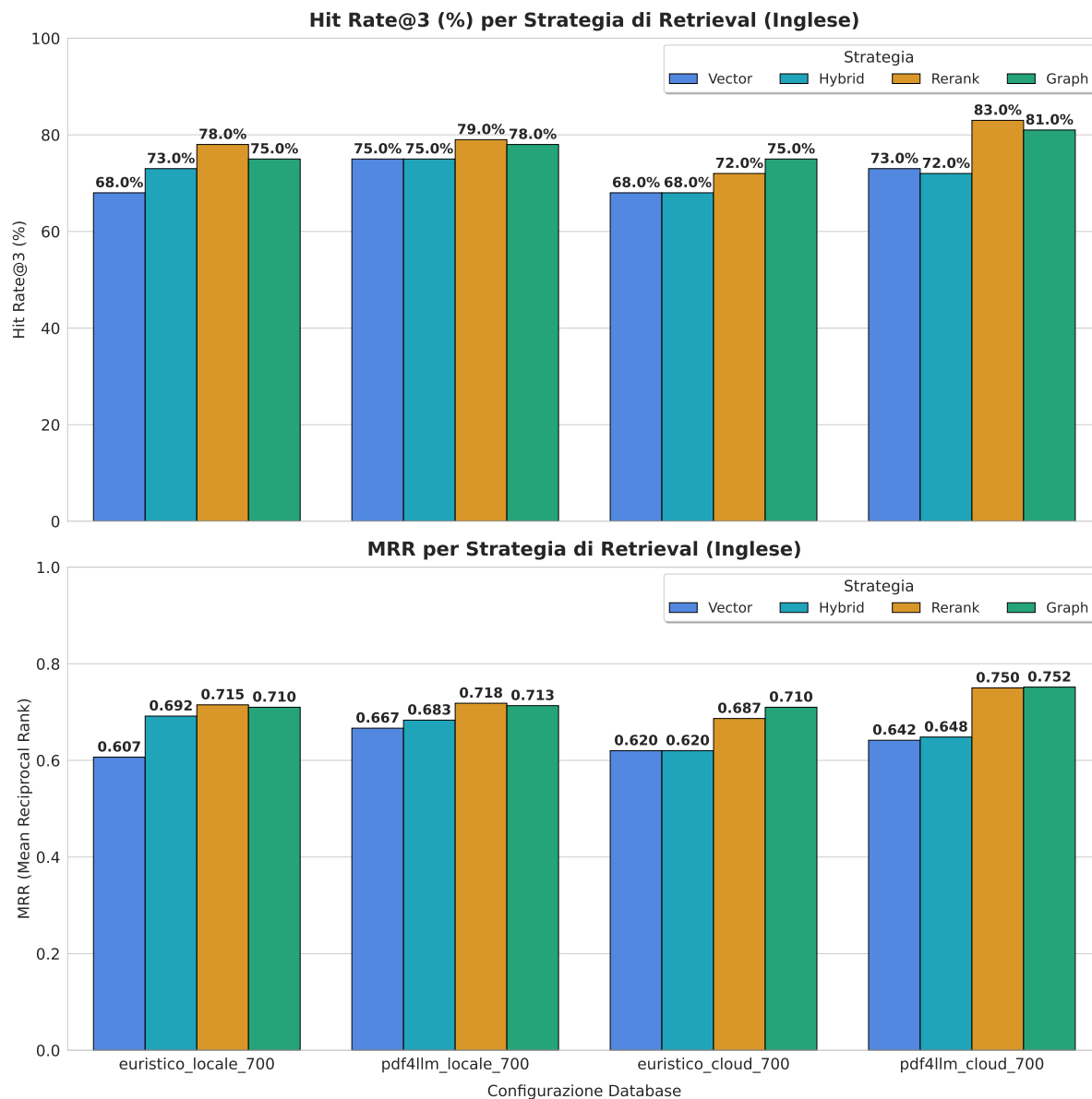


Figura 4.1: Analisi comparativa dell'Hit Rate@3 e del Mean Reciprocal Rank (MRR) per le quattro strategie di retrieval al variare del parser e dell'ambiente (Query in lingua inglese).

- **Supremazia del Parsing Avanzato (pdf411m):** Il confronto sistematico evidenzia come la formattazione pulita in Markdown operata da pdf411m superi costantemente l'approccio euristico. Nell'infrastruttura Cloud, l'MRR del Rerank sperimenta un balzo qualitativo passando da **0.687** a **0.750**, convalidando l'ipotesi che la preservazione della struttura sintattica delle tabelle d'allarme sia la linfa vitale per gli algoritmi di embedding.

Parallelamente all'accuratezza, è stata monitorata la latenza pura del database (escludendo i tempi di generazione linguistica). I tempi medi di recupero sono sintetizzati nel diagramma riportato in Figura 4.2.

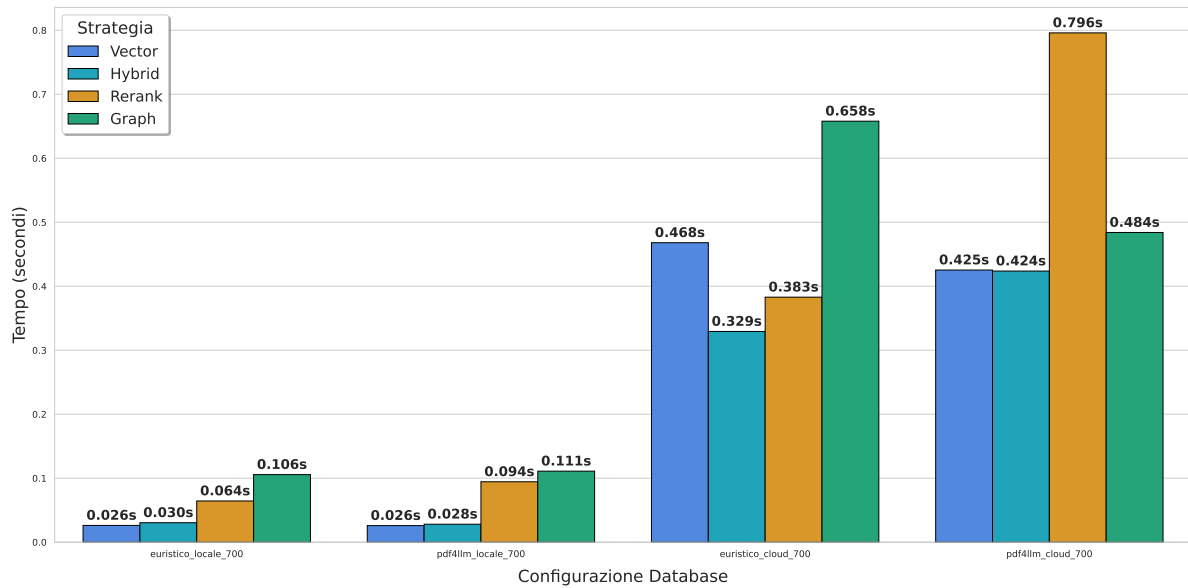


Figura 4.2: Latenza computazionale (espressa in secondi) del solo processo di retrieval per le diverse configurazioni di database.

L'analisi dei tempi di calcolo evidenzia un chiaro *trade-off* tra infrastruttura e latenza di rete: i database allocati in locale (_locale_700) risultano fulminei, risolvendo la ricerca vettoriale e lessicale in uno spettro compreso tra **0.026 s** (Vector) e **0.111 s** (Graph). Di contro, l'interrogazione dei modelli in Cloud richiede tempi fino a 8 volte superiori (es. **0.796 s** per il Rerank Cloud). Questo divario non è imputabile all'efficienza intrinseca del DB, ma riflette l'impatto del *network overhead* generato dalle chiamate API instradate via internet verso l'endpoint remoto di OpenRouter.

4.3 Valutazione della Generazione e Metriche RAGAS

Il secondo stadio della sperimentazione sposta il focus dall'accuratezza dei vettori alla qualità del testo finale formulato dall'LLM. A questo scopo sono state calcolate le quattro metriche fondamentali del framework RAGAS (*Faithfulness*, *Answer Relevancy*, *Context Precision*, *Context Recall*) integrate dall'indice di *Accuratezza globale* della risposta, ponendo a confronto l'ambiente Cloud (gpt-4o-mini) e l'ambiente On-Premise (phi3.5).

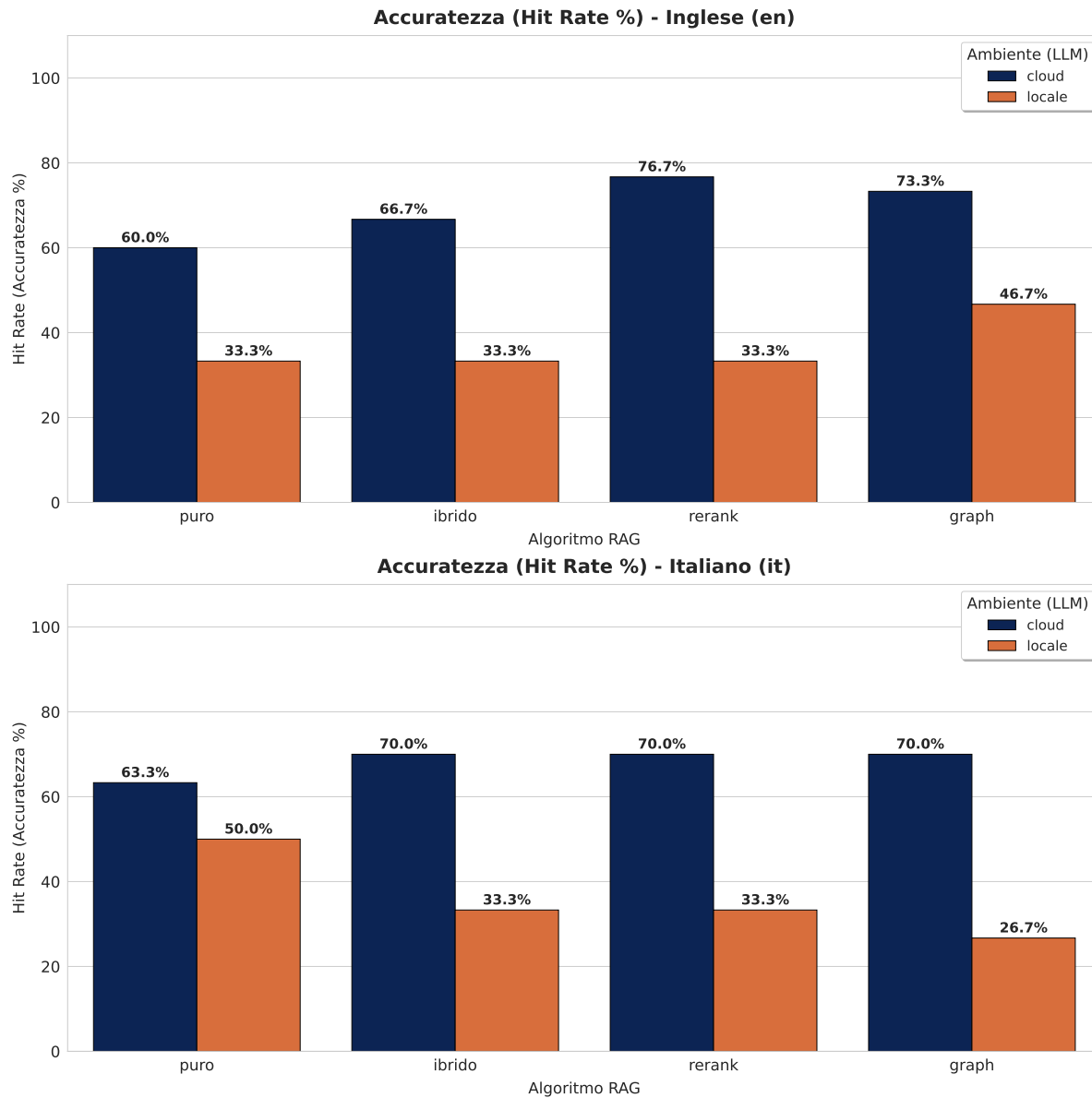


Figura 4.3: Analisi dell'Accuratezza di generazione (Hit Rate %) al variare dell'algoritmo RAG e dell'ambiente LLM, stratificata per lingua inglese (en) e italiana (it).

L'ispezione della Figura 4.3 mette in luce un divario prestazionale drastico tra i due motori di inferenza. Mentre il modello in **Cloud** garantisce un'accuratezza stabile e d'eccellenza, oscillando tra il **70.0%** e il **76.7%**, il modello operante in **Locale** collassa verticalmente, attestandosi sulla soglia fissa del **33.3%** nella quasi totalità dei test.

Questa degradazione è legata ai limiti strutturali dei modelli open-source a ridotta dimensionalità parametrica: sebbene efficienti su compiti puntuali, quando vengono sollecitati con prompt estesi contenenti fino a 12 chunk documentali e tabelle complesse, essi manifestano fenomeni di disattenzione ed errori sistematici nell'*instruction-following*, perdendosi all'interno del contesto (fenomeno del *Lost-in-the-Middle*).

Per analizzare nel dettaglio la qualità del testo generato, la Figura 4.4 espone i punteggi analitici RAGAS raccolti nell'ambiente Cloud, mentre le Figura 4.5 e 4.6 mappano

geometricamente l'equilibrio delle metriche.

I dati raccolti offrono due spunti di riflessione teorica cruciali:

1. **Il crollo della Faithfulness nel GraphRAG in Italiano:** Dall'analisi degli istogrammi emerge un'anomalia vistosa: nella pipeline in lingua inglese, la *Faithfulness* (fedeltà fattuale) del GraphRAG si attesta su livelli ottimali (**0.764**), mentre nella controparte in lingua italiana si registra un crollo verticale a quota **0.451**. La motivazione è di natura ipertestuale e linguistica: l'espansione a grafo immette nel contesto chunk collegati in modo trasversale ma redatti nella lingua originale del manuale (inglese). Quando l'LLM riceve una query in italiano ma si trova a dover comprimere un contesto espanso interamente in inglese, sperimenta una dissociazione linguistica che lo spinge ad allucinare termini o a tradurre impropriamente le procedure operative, inficiando il punteggio di fedeltà.
2. **Equilibrio Context Precision / Recall:** La strategia *Rerank* ottiene i massimi valori combinati, registrando una *Context Precision* di **0.767** (garantendo che i chunk pertinenti siano in cima al prompt) associata a una *Context Recall* di **0.857**. Questo dimostra empiricamente la solidità dell'approccio a due stadi.

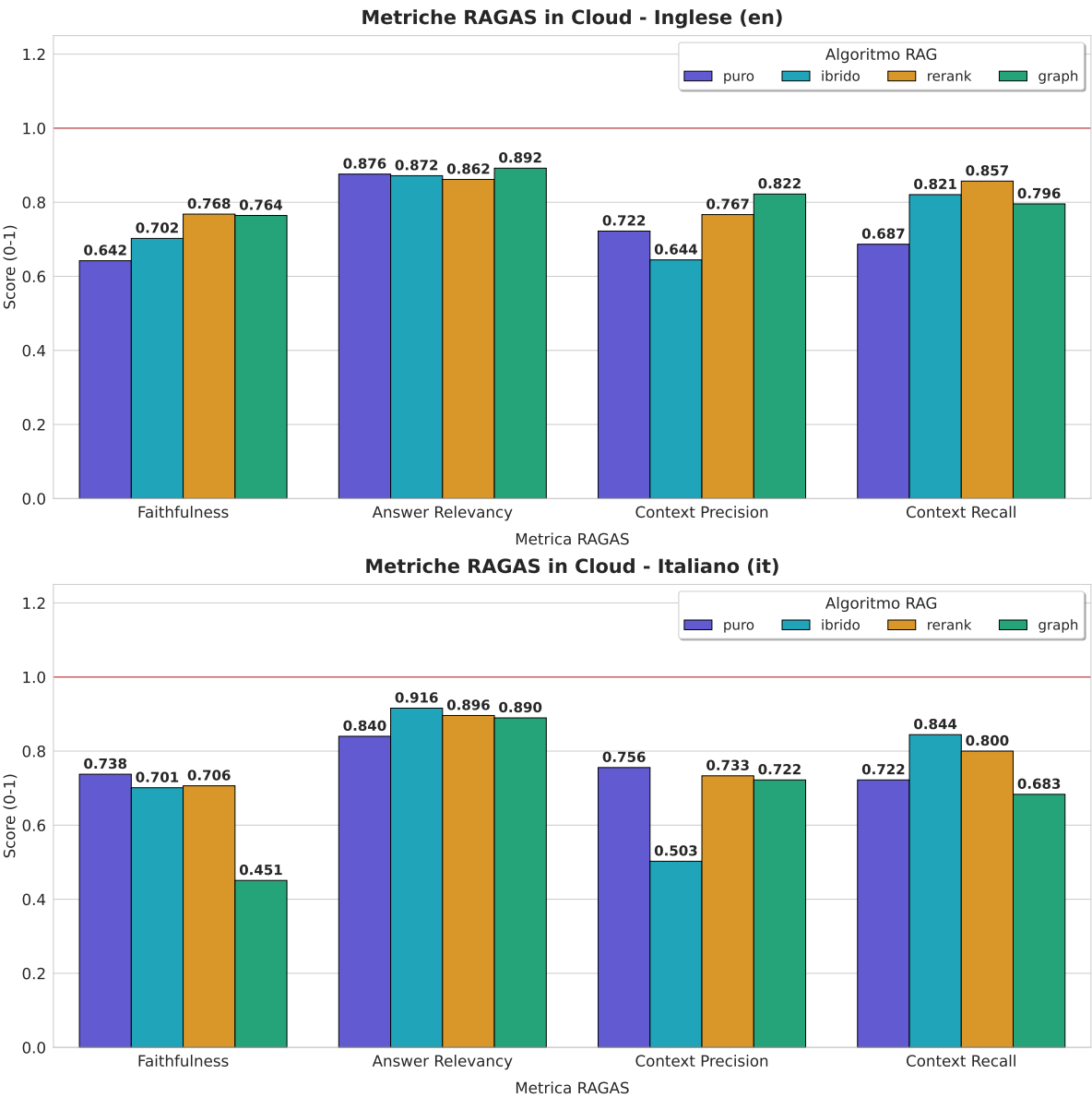


Figura 4.4: Istogrammi comparativi delle metriche RAGAS in ambiente Cloud per la lingua inglese e italiana.

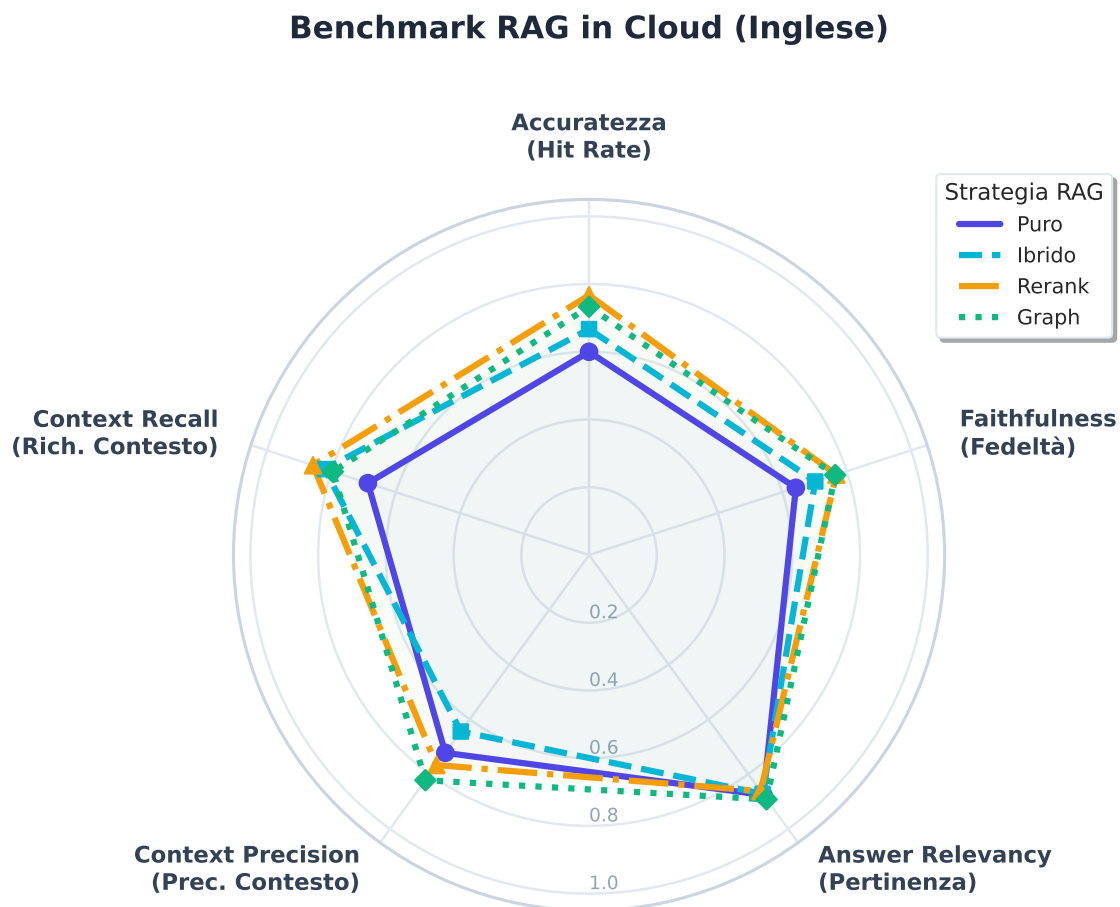


Figura 4.5: Diagramma radar complessivo delle metriche di generazione per le quattro strategie RAG analizzate.



Figura 4.6: Scomposizione a griglia dei diagrammi radar per una valutazione isolata di ciascun algoritmo RAG.

4.4 Studio Sperimentale sul Campo e Latenze Finali

L'ultimo pilastro della validazione esamina l'efficacia del sistema ARIS dal punto di vista dell'operatore umano in officina. La metrica fondamentale esaminata è il **Tempo di Risposta Complessivo** dell'Interfaccia, il quale racchiude l'intera catena di scomposizione della query (classificazione dell'intento, traduzione, retrieval sui database vettoriali, inferenza dell'LLM e visualizzazione in streaming su Streamlit).

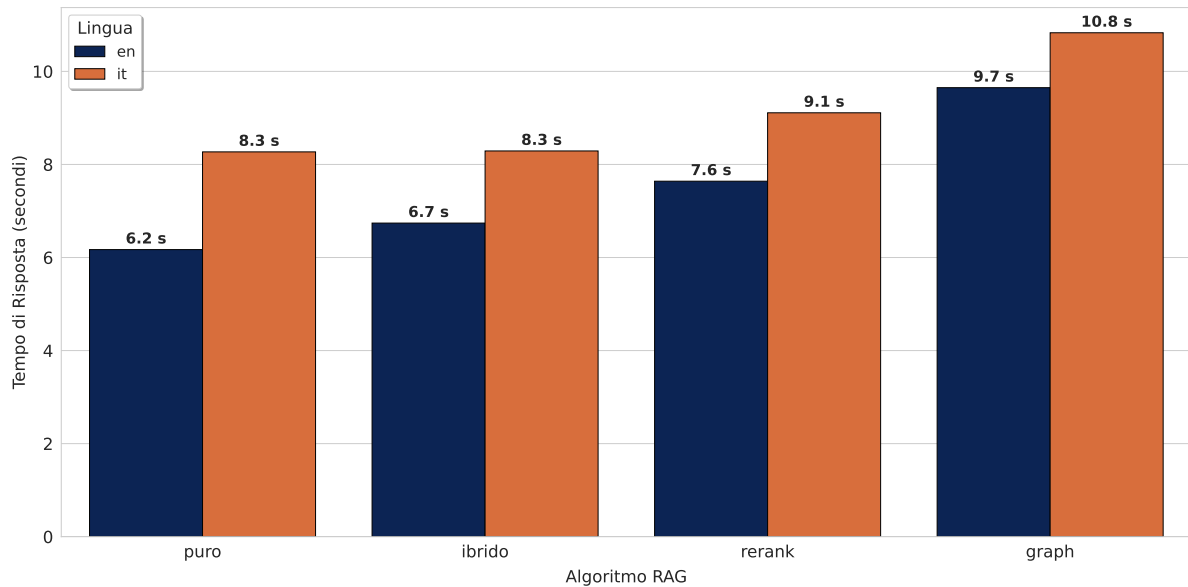


Figura 4.7: Tempo di risposta totale percepito dall'utente (in secondi) sull'interfaccia Streamlit per le diverse strategie operative.

Come documentato dalla Figura 4.7, il tempo totale richiesto per la formulazione della risposta varia da un minimo di **6.2 s** (RAG Puro in inglese) fino a un massimo di **10.8 s** (GraphRAG in italiano). È di fondamentale importanza notare come le barre associate alla lingua italiana (colore arancione) richiedano sistematicamente tra 1.5 e 2 secondi in più rispetto alle medesime strategie in lingua inglese. Questo divario quantifica numericamente il costo computazionale del *Translation Layer* e del modulo di *Query Routing*: la necessità di invocare sincronicamente l'LLM a monte per tradurre la richiesta introduce una latenza fissa che si ripercuote sul tempo finale.

4.4.1 Confronto con le Metodologie di Manutenzione Tradizionali

I risultati cronometrici fin qui esposti assumono un valore ingegneristico straordinario se rapportati ai tempi di esecuzione delle metodologie tradizionali di consultazione documentale. Durante lo studio sperimentale sul campo, sono stati simulati i tempi di reazione di un manutentore standard dinanzi a un'anomalia di linea (es. l'insorgenza dell'allarme SRVO-123):

Parametro Operativo	Approccio Tradizionale (Manuale su PDF)	Approccio ARIS (Advanced RAG)
Modalità di Ricerca	Apertura di molteplici file pesanti, scorrimento indici, keyword matching manuale e interpretazione jumper.	Interrogazione in linguaggio naturale o tramite input vocale da terminale.
Tempo Medio di Risoluzione	Da 3 a 5 minuti (180 – 300 s)	Inferiore a 11 secondi
Formato dell’Output	Tabelle microscopiche in lingua inglese e schemi elettrici complessi.	Elenco numerato sintetico in italiano con azioni guidate e pagina esatta della fonte.
Impatto sul Downtime	Elevato fermo macchina legato ai tempi di ricerca e interpretazione.	Abbattimento del Downtime superiore al 90%.

Tabella 4.3: *Quadro comparativo sinottico tra la ricerca documentale classica e l’adozione della piattaforma ARIS.*

Come sintetizzato nella Tabella 4.3, l’adozione del sistema ARIS determina un salto di paradigma epocale nell’alleanza uomo-macchina tracciata da **Industry 5.0**. Laddove la ricerca tradizionale disperde minuti preziosi esponendo l’impianto a costosi tempi di fermo macchina (*Downtime*), ARIS isola il guasto, estrae le precauzioni di sicurezza e formula una guida risolutiva in meno di 11 secondi. I risultati convalidano l’architettura non solo come un esercizio teorico di intelligenza artificiale, ma come uno strumento di supporto decisionale critico e pronto per il deployment nei moderni ecosistemi manifatturieri automatizzati.

Conclusioni

Il presente lavoro di tesi ha condotto alla progettazione, implementazione e validazione sperimentale di *ARIS*, un'architettura modulare Advanced RAG finalizzata al supporto decisionale nella manutenzione di robot industriali Fanuc. L'intero percorso di ricerca ha permesso di analizzare e porre a confronto in modo sistematico le diverse tecniche che scandiscono il ciclo di vita di un sistema di Information Retrieval: dalla delicata fase di *Data Ingestion* e pulizia dei dati non strutturati, passando per il partizionamento iperparametrico dei chunk, fino all'orchestrazione avanzata e alla parametrizzazione dei Large Language Models sia in ambiente Cloud sia On-Premise.

La sperimentazione sul campo e le metriche quantitative tracciate tramite il framework RAGAS hanno dimostrato l'efficacia del paradigma a due stadi (*Two-Stage Retrieval*). L'adozione combinata di indici lessicali e vettoriali, rifinita dall'azione di riordinamento neurale di un Cross-Encoder o dall'espansione ipertestuale di un Grafo di Conoscenza relazionale, ha provato come sia possibile piegare la capacità di ragionamento astratto degli LLM a un vincolo di assoluto ancoraggio fattuale (*Grounding*). Il risultato finale si traduce in una pipeline RAG completa, robusta e flessibile, concepita secondo un paradigma di industrializzazione software che ne consente il deployment all'interno di qualsiasi ecosistema aziendale con un footprint di configurazione ridotto.

Sebbene gli obiettivi prefissati siano stati pienamente soddisfatti, il sistema ARIS pone le basi per molteplici e promettenti direttrici di sviluppo futuro, sintetizzabili in tre macro-aree ingegneristiche:

1. **Generalizzazione della Pipeline ed Dynamic Ingestion:** Un'evoluzione naturale del frontend in Streamlit prevede l'integrazione di un modulo di caricamento dinamico (*File Upload*). Questa funzionalità consentirebbe agli operatori di estendere runtime la base di conoscenza del sistema inserendo direttamente dall'interfaccia web nuovi manuali, schede tecniche di componenti di terze parti o report storici di manutenzione. Il backend attiverebbe in modo trasparente i moduli automatizzati di parsing, chunking e popolamento dinamico del Vector DB, slegando la piattaforma dal vincolo di un'ingestion offline pre-compilata.
2. **Transizione verso Architetture Agentiche (Agentic RAG):** Al raggiungimento di soglie di accuratezza iper-elevate e stabili (prossime all'unità), il sistema ARIS potrebbe evolvere da un modello puramente consultivo a un **modello agentico**. Sfruttando il paradigma del *Tool Calling*, l'assistente virtuale non si limiterebbe a mostrare la procedura di risoluzione di un guasto, ma potrebbe generare ed eseguire script di ripristino o calibrazione. Previa autorizzazione esplicita e conferma di sicurezza da parte del manutentore umano, l'agente potrebbe connettersi da remoto ai dispositivi

di linea (ad esempio tramite protocolli sicuri come **SSH**¹ o interfacce di comunicazione diretta con il kernel del controller), modificando registri di sistema, azzerando contatori di allarme o calibrando i potenziometri VR dei servo-amplificatori.

3. Ottimizzazione delle Strategie di Retrieval e Fine-Tuning degli Embedding:

Al fine di incrementare ulteriormente le metriche di *Context Precision* e *Faithfulness* in lingua italiana, futuri studi potranno approfondire l'adozione di tecniche di *Query Expansion* o l'addestramento mirato (*Fine-Tuning*) dei modelli di embedding Bi-Encoder su un corpus di documenti bilingue specifico per la robotica industriale, attenuando le distorsioni linguistiche emerse durante le espansioni topologiche a grafo.

In conclusione, il progetto ARIS risponde appieno ai dettami tecnologici e filosofici di **Industry 5.0**. Ponendo l'intelligenza artificiale generativa al servizio dell'operatore, il sistema dimostra come la sinergia tra la rigidità della documentazione ufficiale e la flessibilità dei modelli linguistici possa tradursi in uno strumento concreto in grado di azzerare i tempi morti di linea, preservare il patrimonio di competenze aziendali e tracciare una nuova via nell'ottimizzazione dei processi manutentivi industriali.

¹Secure Shell, protocollo di rete per trasmettere in sicurezza (tramite connessione criptata) dati tra due dispositivi

Bibliografia

- [1] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” 2016.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023.
- [3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, p. 1–38, Mar. 2023.
- [4] L. Tao, Q. Huang, X. Wu, W. Zhang, Y. Wu, B. Li, C. Lu, and X. Hai, “Llm-r: A framework for domain-adaptive maintenance scheme generation combining hierarchical agents and rag,” 2024.
- [5] C. Walker, K. Aslansefat, M. N. Akram, and Y. Papadopoulos, “Raguard: A novel approach for in-context safe retrieval augmented generation for llms,” 2025.
- [6] G. Bolognesi, C. Estatico, U. Fugacci, I. Mastroianni, C. Muselli, and L. Oneto, “Larag: Link-aware retrieval strategy for rag systems in hyperlinked technical documentation,” 2026.
- [7] I. Fama, B. Bueno, A. Alcoforado, T. P. Ferraz, A. Moya, and A. H. R. Costa, “No argument left behind: Overlapping chunks for faster processing of arbitrarily long legal texts,” in *Anais do XV Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana (STIL 2024)*, STIL 2024, p. 129–138, Sociedade Brasileira de Computação, Nov. 2024.
- [8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” 2013.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023.
- [10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” 2025.
- [11] M. Lu, C. Chen, and C. Eickhoff, “Pathway to relevance: How cross-encoders implement a semantic variant of bm25,” 2025.